

高性能计算导论

第10讲：并行优化

翟季冬
计算机系

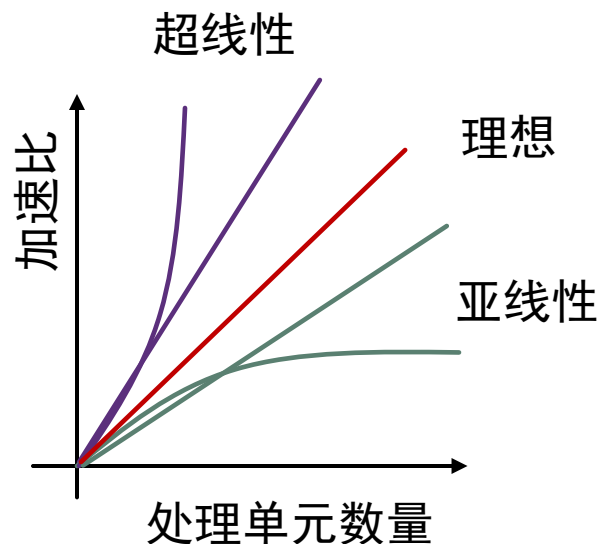
目录

- 并行指标
- 任务分配
- 消息传递
- 通信优化
- 通信开销
- 通信拥塞
- 可扩展性

并行计算性能指标

加速比

- **加速比**: $S(p) = \frac{T_S}{T_p}$
 - T_S : 程序的串行版本执行时间
 - T_p : 程序的并行版本执行时间, 使用的处理单元数目为 p
- **线性加速**: $S(p) = p$
 - 理论中的理想最大加速比
- **超线性加速**: $S(p) > p$
 - 实际中有时发生
 - 例如: 数据缓存在cache
- **亚线性加速**:
 - 典型加速情况



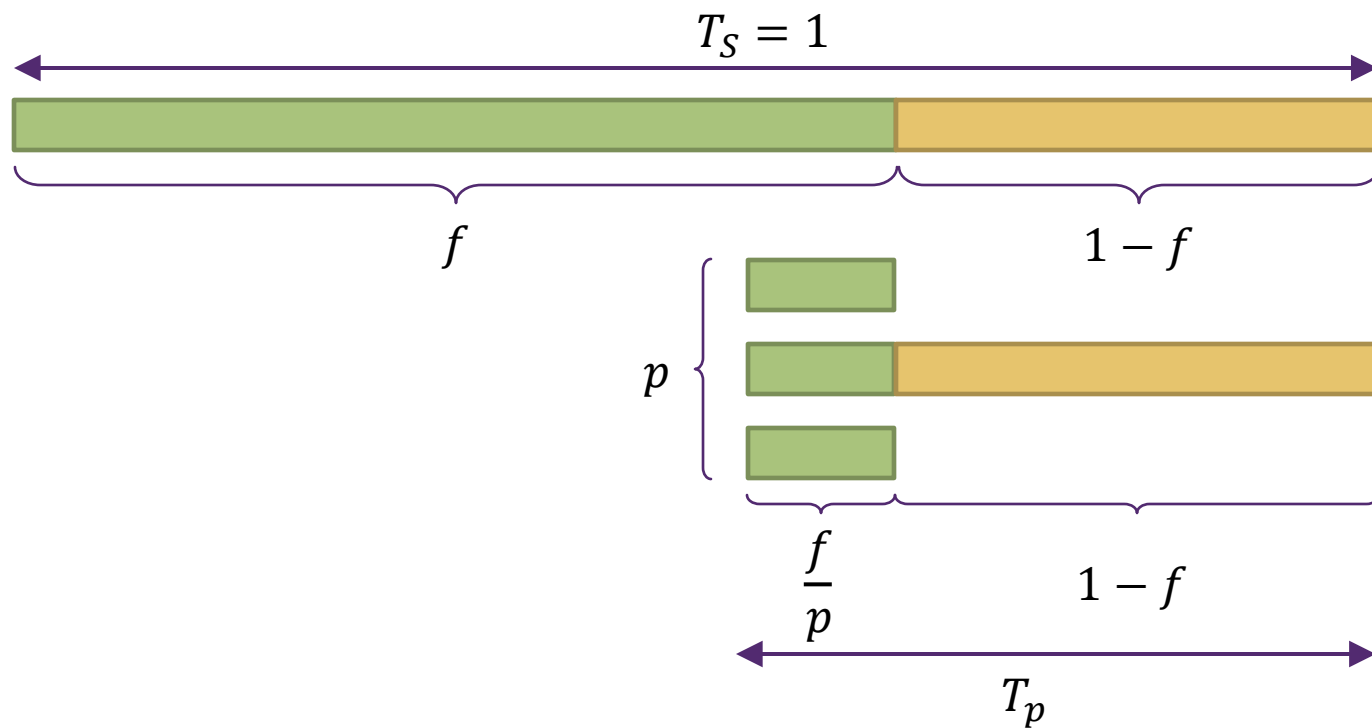
理想线性加速

- 难以实现理想线性加速的原因
 - 程序中不是所有部分都可以并行
 - 导致处理器闲置
 - 并行处理引入通信和同步开销
 - 通常是主要因素
 - 并行处理可能引入额外计算
 - 为了减轻通信和同步开销，可能会做少量重复计算

Amdahl's law-阿姆达尔定律

- 描述理想情况下，程序的加速比与可并行部分的关系
- 记可并行部分比例为 f ，使用的处理单元个数为 p

$$S(p) = \frac{T_S}{T_p} = \frac{1}{(1-f) + \frac{f}{p}}$$

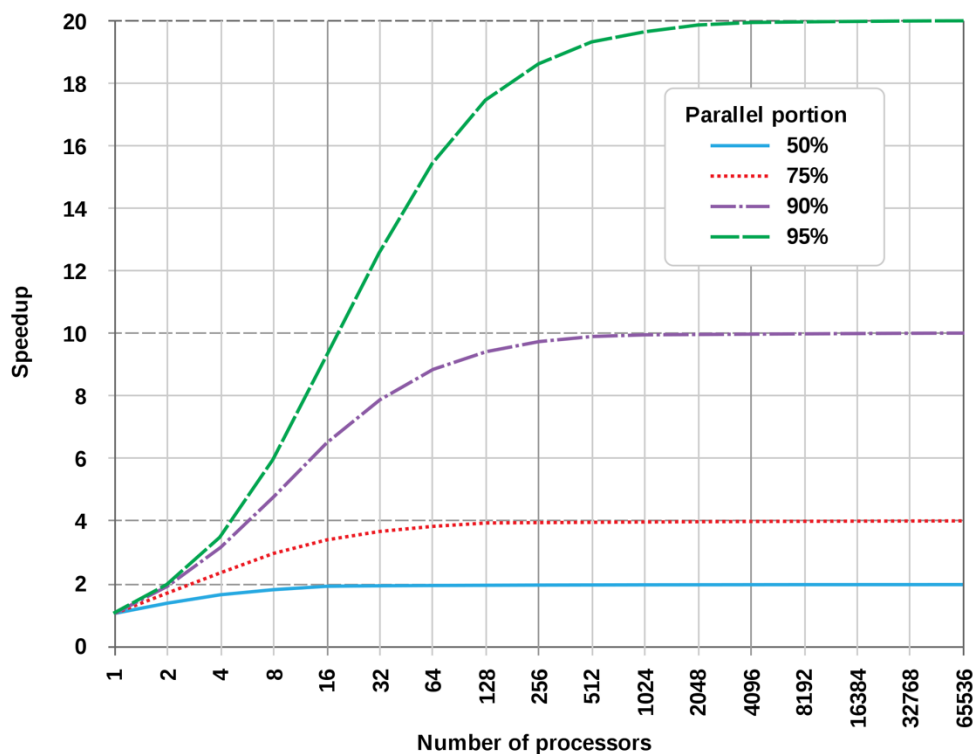


Amdahl's law-阿姆达尔定律

- 描述理想情况下，程序的加速比与可并行部分的关系
- 记可并行部分比例为 f ，使用的处理单元个数为 p **1-f: 串行部分**

$$S(p) = \frac{T_s}{T_p} = \frac{1}{(1-f) + \frac{f}{p}}$$

$$S(p) < \lim_{p \rightarrow \infty} S(p) = \frac{1}{1-f}$$



并行度概念

强扩展与弱扩展

- 强扩展（Strong scaling）：

问题规模一定的情况下，增加处理单元的数目

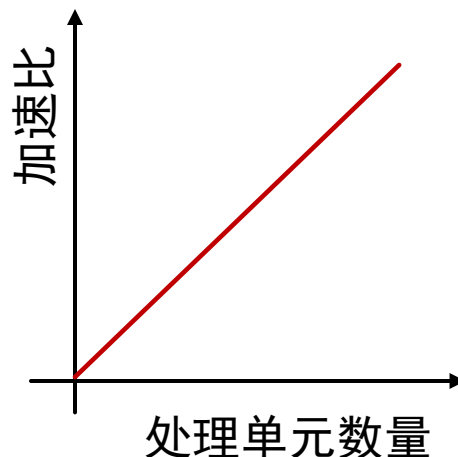
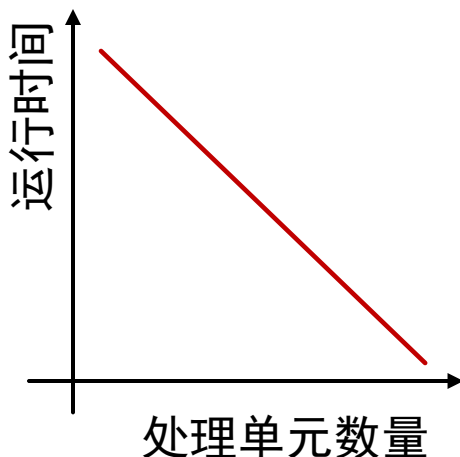
- 通常用于寻找资源使用的“最优点”

- 在合理的时间内完成计算，但不会由于并行开销而浪费太多资源

- 加速比的定义是基于强扩展定义的

- 理想线性加速比： $S(p) = p$

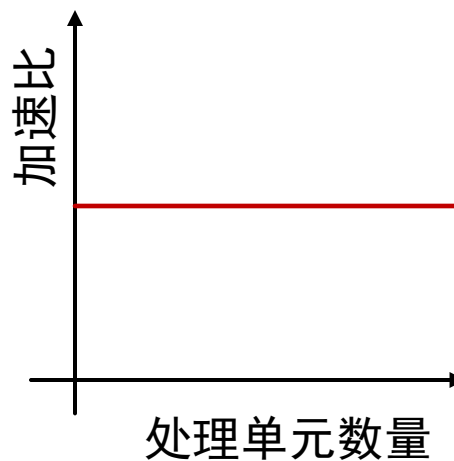
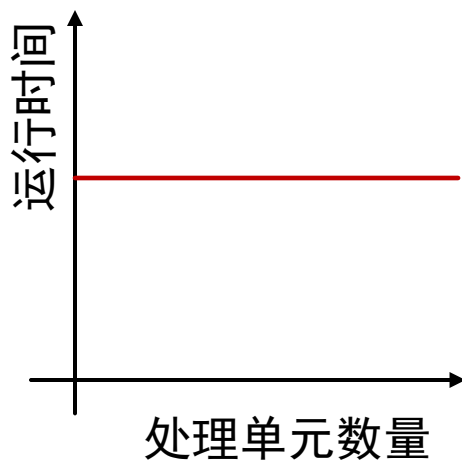
理想线性加速情况



强扩展与弱扩展

- 弱扩展（weak scaling）：
 - 问题规模正比于处理单元数量
- 用于求解更大规模的问题
- 理想线性加速比： $S(p) = \text{定值}$

理想线性加速情况



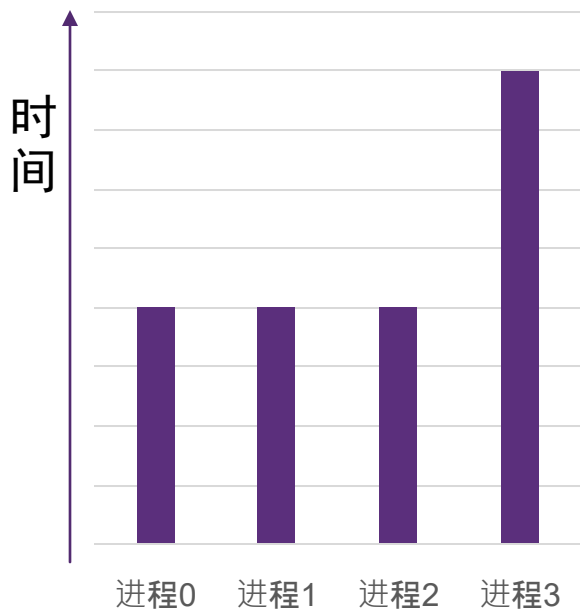
任务分配

并行程序性能优化

- 程序的并行化过程：
 - 问题拆分
 - 任务分配
 - 进程组织
- 并行优化的目标
 - 改进负载均衡
 - 减少通信操作
 - 避免资源拥塞
 - 减少额外开销

负载均衡

- **理想情况**：所有处理器在运行程序的整个时间段内都在满负荷工作
 - 一直进行计算
 - 同时完成工作
- 很小的负载不均衡也可能会**严重影响加速比**



负载不均衡程序

- 进程 3 做了 2 倍的工作
- ➔ 进程 3 需要 2 倍的时间
- ➔ 并行程序的 **50% 时间**消耗在串行执行上

(串行部分的工作量是总工作量的约 20%，
即在 Amdahl 定律中 $S=0.2$ ，**最大加速比 5**)

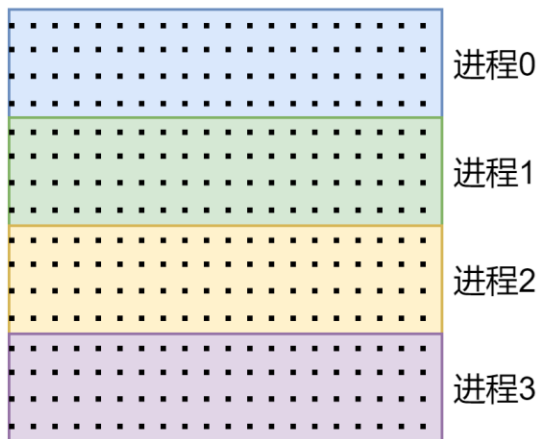
任务分配

- 任务分配方式：
 - 静态分配
 - 半静态分配
 - 动态分配

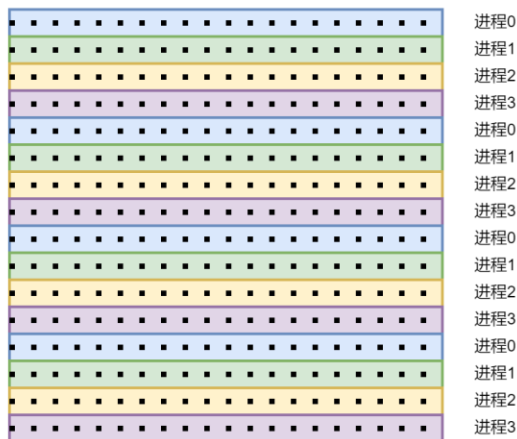
静态分配 - Static assignment

- 静态分配：
 - 任务分配不依赖于运行时的因素（如某一个任务的执行时间）
 - 可以在编译时确定
 - 分配算法也可以考虑输入相关参数：输入数据大小、线程数
- 静态分配优点：简单，分配几乎没有额外开销
- 例子：将相同数量的网格分配给每个进程
 - 两种静态的任务分配方式：块状划分、交错划分

块状划分

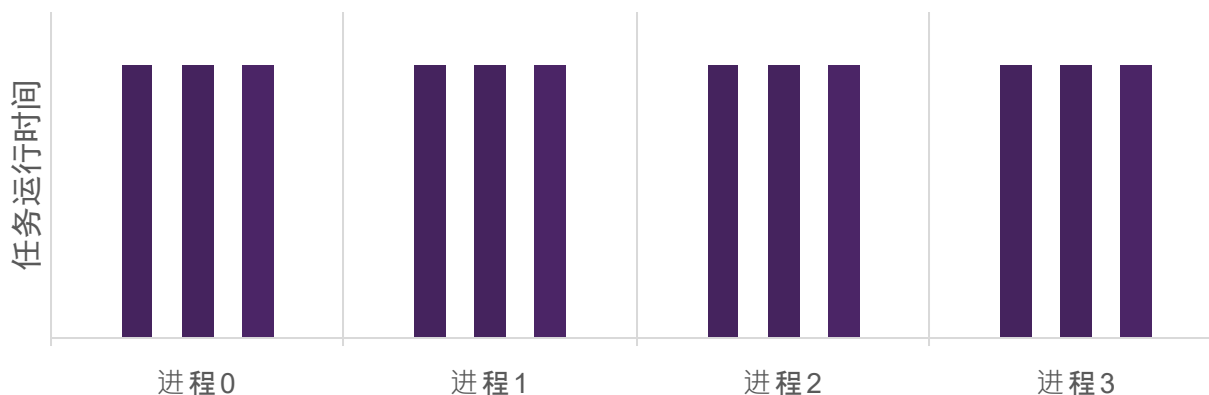


交错划分



何时使用静态分配-1

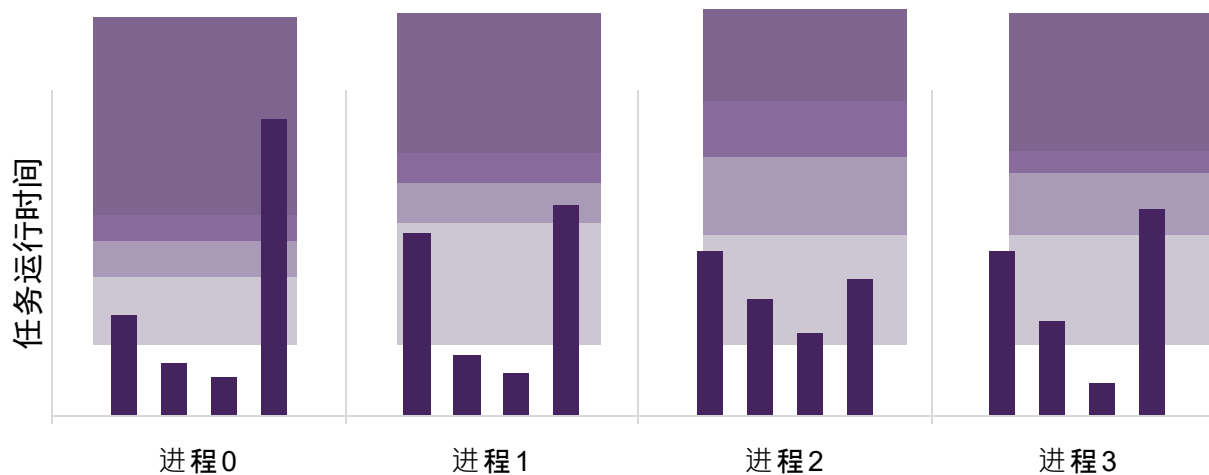
- 当负载的开销（运行时间）和工作量是**可预测的**
 - 可以在工作开始前**预先计算好**分配方案
- 例子：所有任务的耗时都是**相同的**



- 12 个任务，每个任务的开销是相同的
- **分配方案**：静态地分配给 4 个处理器，每一个分配 3 个任务

何时使用静态分配-2

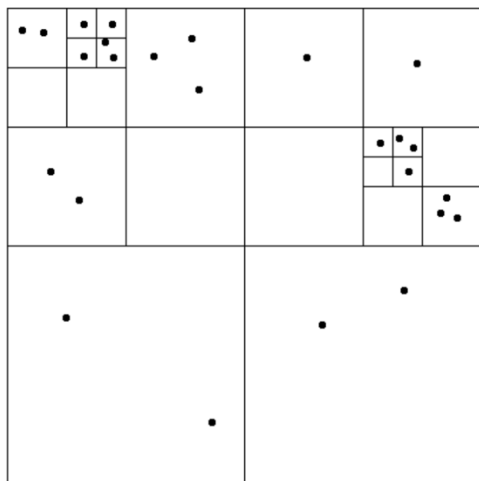
- 当负载的开销（运行时间）和工作量是**可预测的**
- 但并非每个任务开销相同
- 或当运行时间的统计量是**可预计的**（如平均开销）



- 分配策略：
 - 将**总开销相同**的任务分配给每个处理器
 - 确保良好的（平均）负载均衡

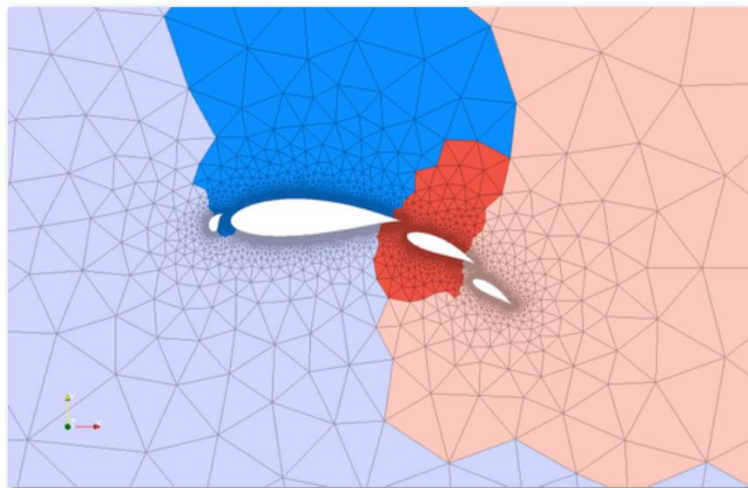
半静态分配 - Semi-static assignment

- 在不远的将来，工作的负载是可预测的
 - **观察**：最近的历史可以用来**预测较近的将来**
- 利用**程序的周期性**分析它的运行状况并重新分配任务
 - 在一个重新分配周期内，分配是“**静态**”的



■ 例子1：粒子模拟

- 在粒子运动过程中根据位置重新划分
- 如果粒子运动较慢，那么重新划分频率可以很低



■ 例子2：自适应网格

- 网格根据物体移动而改变
- 改变发生较慢（不同颜色代表分配到不同处理器上的网格）

动态分配 - Dynamic assignment

- 程序在运行时**动态分配任务**来确保好负载均衡
 - 任务的运行时间或数量是**不可预测**的

顺序程序（迭代之间无关）

```
int N = 1024;
int* x = new int[N];
bool* is_prime = new bool[N];
// 假设这里有 x 的初始化
for (int i = 0; i < N; ++i){
    // 运行时间未知
    is_prime[i] =
        test_primalilty(x[i]);
}
```

并行程序

（多个线程 SPMD 执行，共享内存模型）

```
int N = 1024;
// 假设内存分配仅使用一个线程
int* x = new int[N];
bool* is_prime = new bool[N];
```

// 假设这里有 x 的初始化

```
LOCK counter_lock;
int counter = 0; // 共享变量
```

```
while (1) {
    int i;
    lock(counter_lock);
    i = counter++;
    unlock(counter_lock);
    if (i >= N) break;
    is_prime[i] =
        test_primalilty(x[i]);
}
```

} atomic_inc(counter)

使用任务队列来动态分配

- 问题 →
分解成一系列子任务

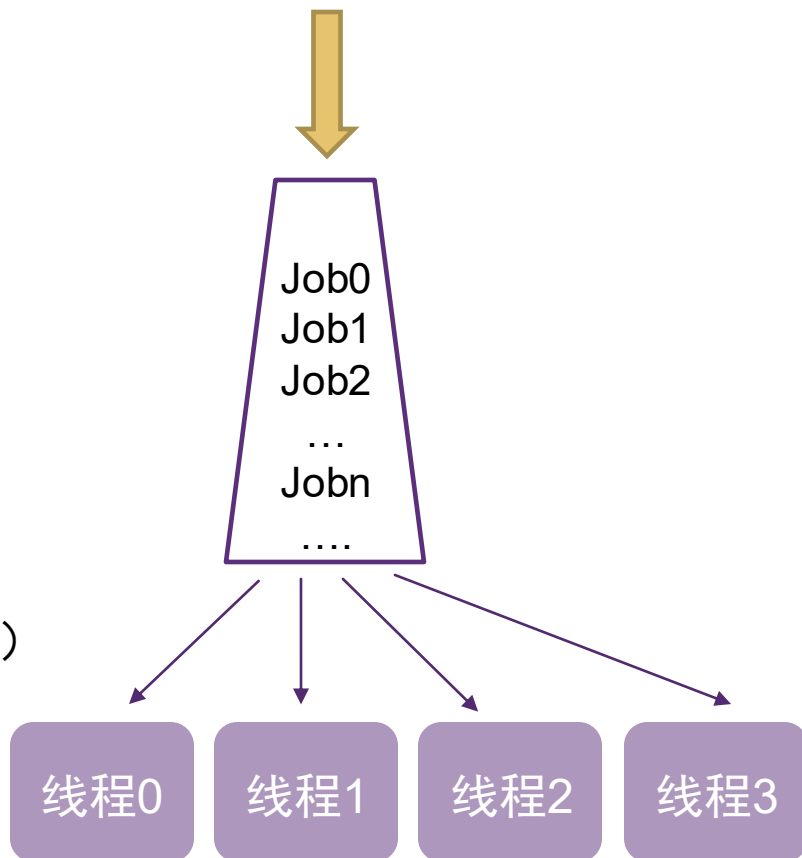


- **共享任务队列：**

- 需要被完成的任务
- 假设每个任务都是独立的

- **工作线程：**

- 从共享队列中拉取任务（pull）
- 将新创建的任务推到队列中（push）



任务粒度- Granularity

- 如下代码的问题是什么？

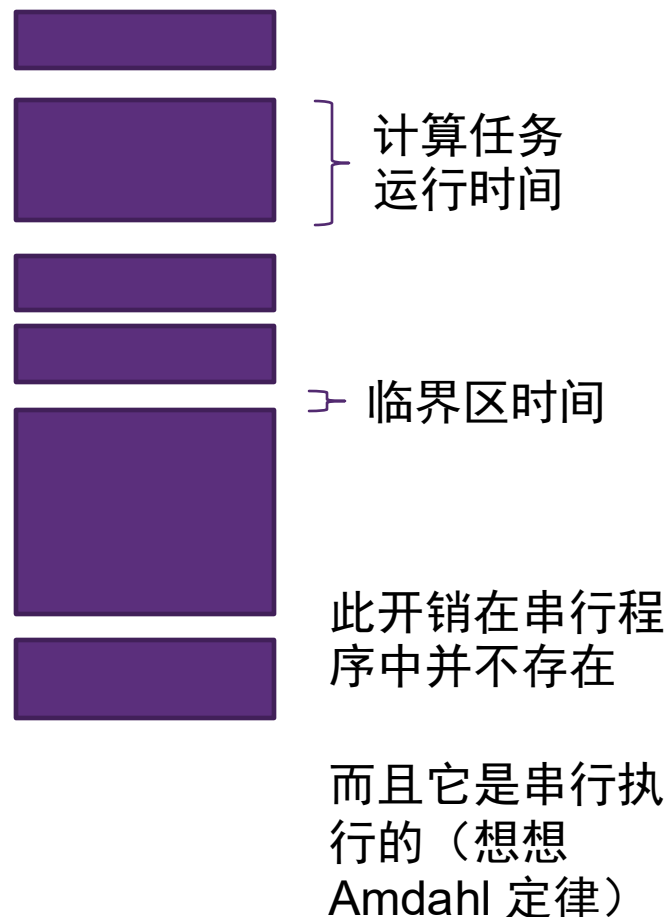
```
int N = 1024;
// 假设内存分配仅使用一个线程
int* x = new int[N];
bool* is_prime = new bool[N];

// 假设这里有 x 的初始化

LOCK counter_lock;
int counter = 0; // 共享变量

while (1) {
    int i;
    lock(counter_lock);
    i = counter++;
    unlock(counter_lock);
    if (i >= N) break;
    is_prime[i] =
        test_primalty(x[i]);
}
```

- 细粒度划分：一个任务 = 一个元素
- 看起来负载均衡很好（很多小任务）
- 问题：同步开销很大（大量临界区串行运行）



增加任务粒度

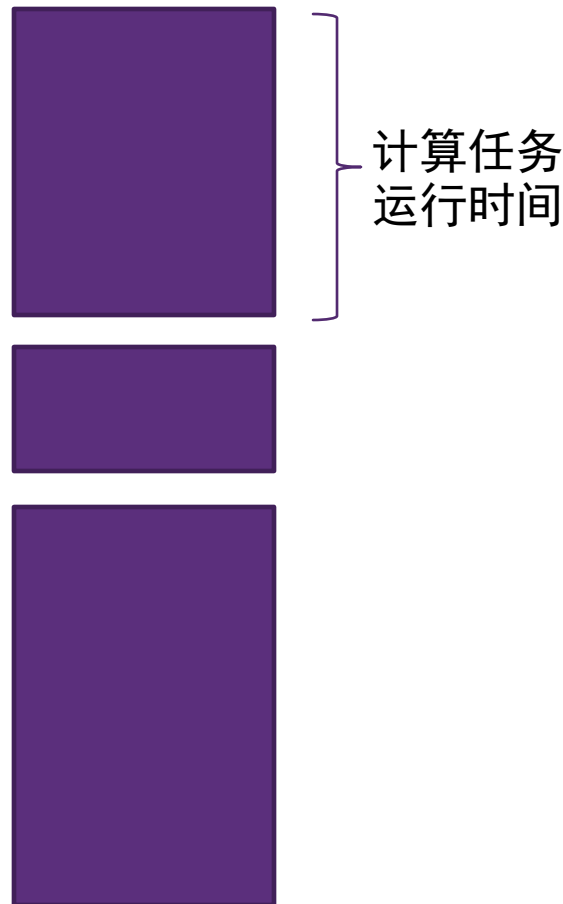
```
int N = 1024;
int GRANULARITY = 10;

// 假设内存分配仅使用一个线程

int* x = new int[N];
bool* is_prime = new bool[N];

// 假设这里有 x 的初始化
LOCK counter_lock;
int counter = 0; // 共享变量

while (1) {
    int i;
    lock(counter_lock);
    i = counter, counter += GRANULARITY;
    unlock(counter_lock);
    if (i >= N) break;
    for (int end = min(i + GRANULARITY, N);
         i < end; ++i) {
        is_prime[i] =
            test_primality(x[i]);
    }
}
```



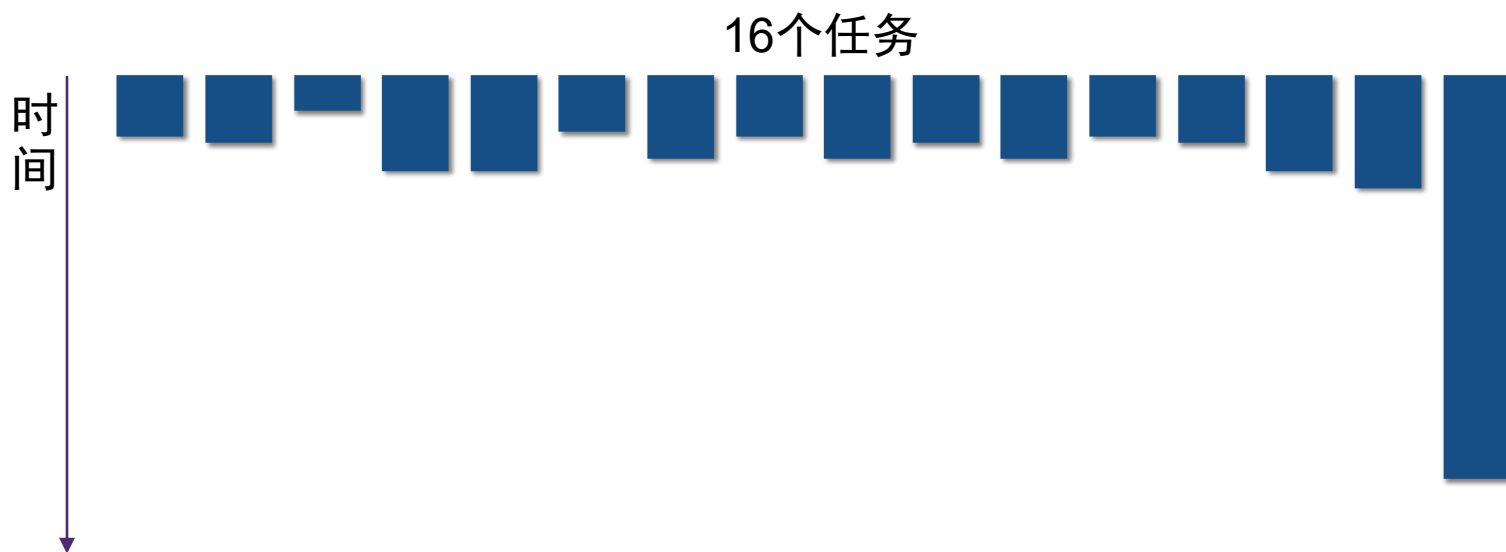
- 粗粒度划分（coarse）：一个任务 = 10 个元素
- 减少同步开销
 - 临界区减少到十分之一

任务粒度考虑

- 任务数量远比处理器数量多时
 - 小任务可以提高**负载均衡**
 - 期望任务粒度更小
- 减少任务数可以减少任务**分配开销**
 - 期望任务粒度更大
- 理想的任务粒度依赖于很多因素
 - 核心：了解**工作负载**和**底层运行系统**

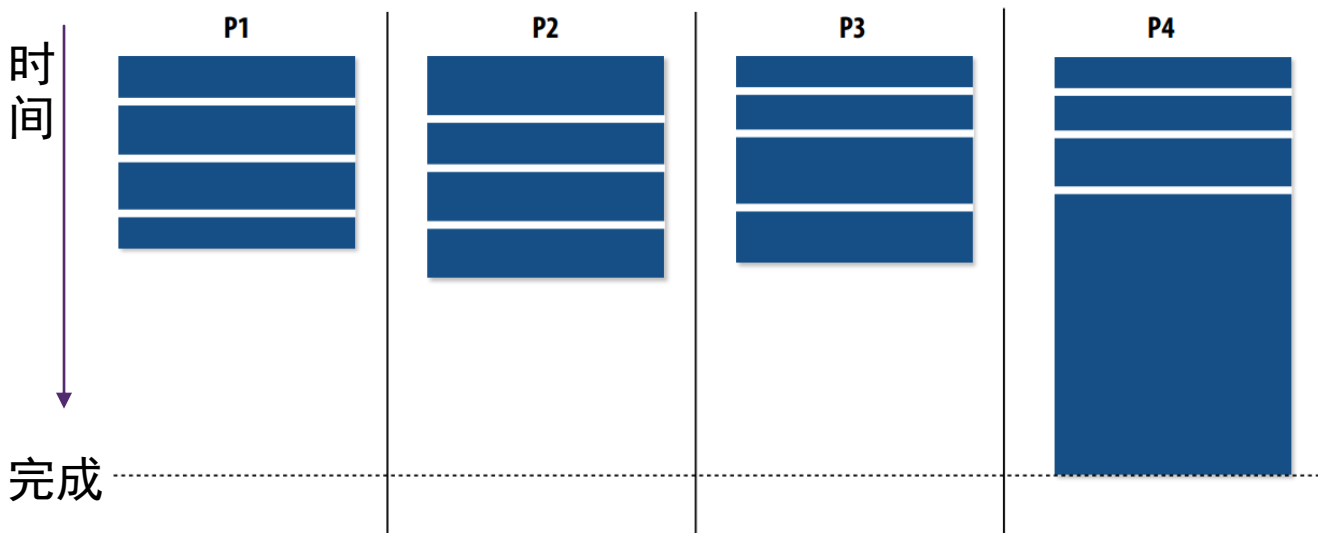
任务分配

- 考虑使用一个共享队列来分配任务
- 从左到右分配这些任务会发生什么？



任务分配

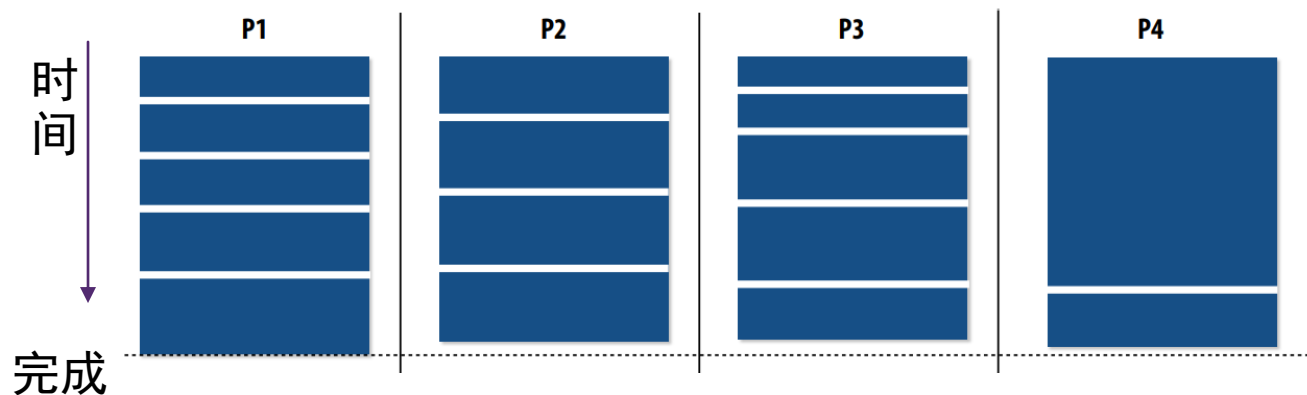
- 最后跑最长的任务会发生什么？
 - 负载不均衡！



- 可能的解决方案：将任务切成更多更小的子任务
 - 耗时最长的任务会相对变得短一些
 - 会增加同步开销
 - 可能长任务不允许切分（必须串行执行）

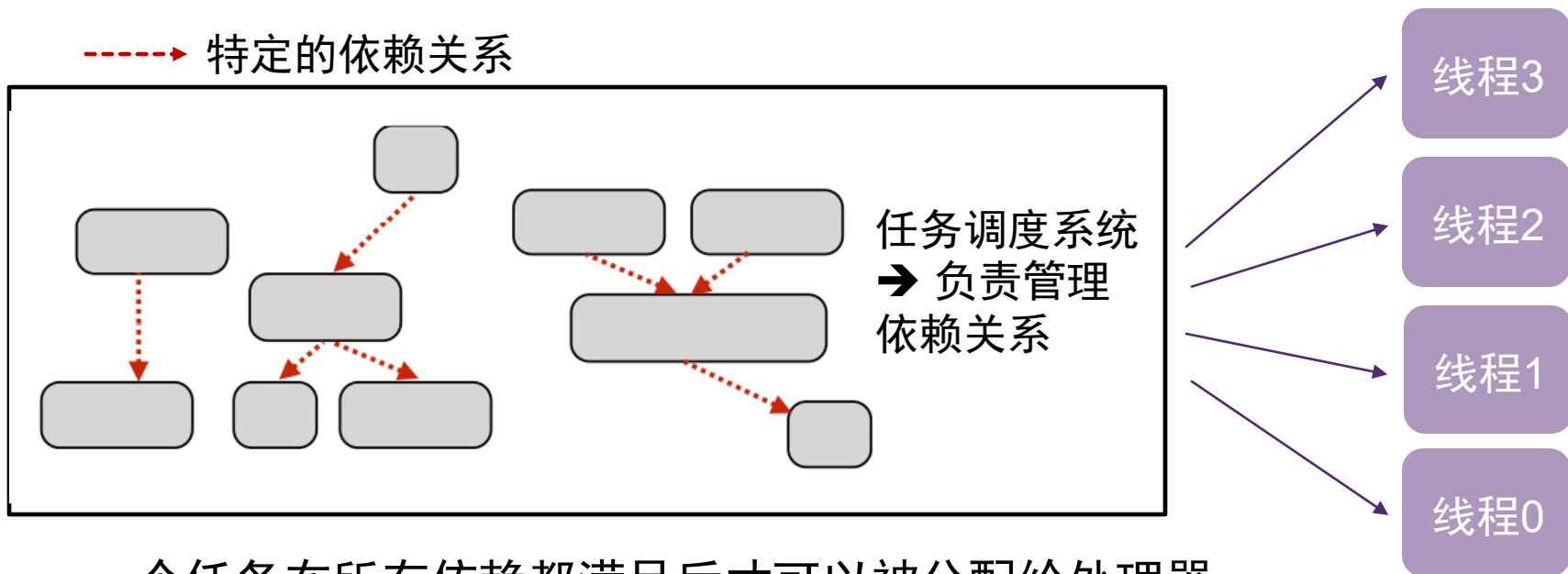
更智能的任务分配

- 先调度长任务，来减少计算最后的“小尾巴”



- 更智能的任务分配
 - 先调度长任务
 - 处理长任务的线程处理的任务总数更少，但是总的工作量和其它线程相近
 - 需要对负载有一定的先验知识

任务之间不一定是独立的



- 一个任务在所有依赖都满足后才可以被分配给处理器
- 处理器可以向调度器提交新的任务：有**显式依赖关系**

```
foo_handle = enqueue_task(foo); // 提交 foo 任务, 无依赖  
bar_handle = enqueue_task(bar, foo_handle); // 提交 bar 任务, 依赖于 foo
```

例子：深度学习程序执行流程

```
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)
z = tf.placeholder(tf.float32)

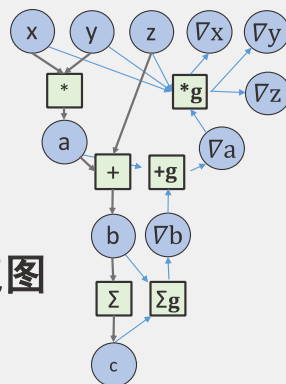
a = x * y
b = a + z
c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    sess.run([grad_z], feed_dict=values)
```

深度学习代码

依赖任务流图



- 体系结构无关的图层优化(图融合、替换、删冗)
- 任务调度



异构芯片



数学算子库,
张量编译等



- 体系结构相关算子优化

小结

- 目标：达到**较好的负载均衡**
 - 希望所有处理器都一直在工作
 - 但是希望解决方案的**开销尽量低**
 - 减少计算开销（比如调度、分配的开销）
 - 减少同步开销
- **静态分配 vs. 动态分配**
 - 两者并非相对概念，有很多折中选择
 - 尽量地运用负载知识来减少负载不均衡和任务管理及同步的开销
 - **理想情况**：如果知道所有信息，则使用静态分配策略

优化例子

稀疏矩阵向量乘法 - SpMV

- 一个稀疏矩阵 A 和一个稠密向量 x 相乘
- 获得一个稠密向量 y , 即 $y = Ax$

		1	
2	3		
4		5	6

A
(4x4)

稀疏矩阵

x

a
b
c
d

x
(4x1)

稠密向量

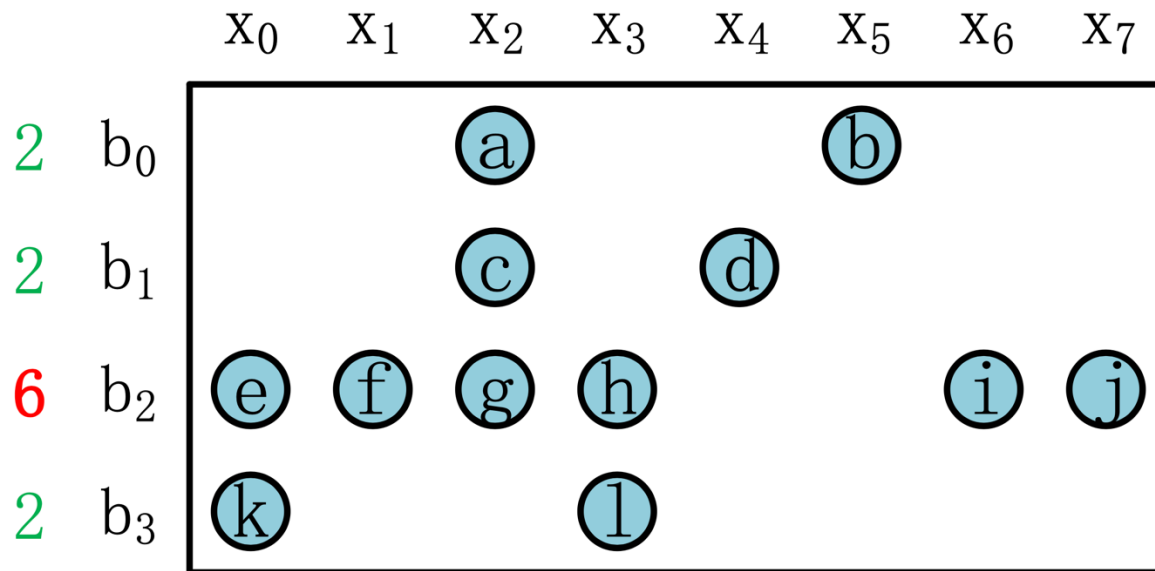
=

$1c$
$2a+3b$
0
$4a+5c+6d$

y
(4x1)

稠密向量

稀疏矩阵存储格式



坐标列表 COO (Coordinate list)

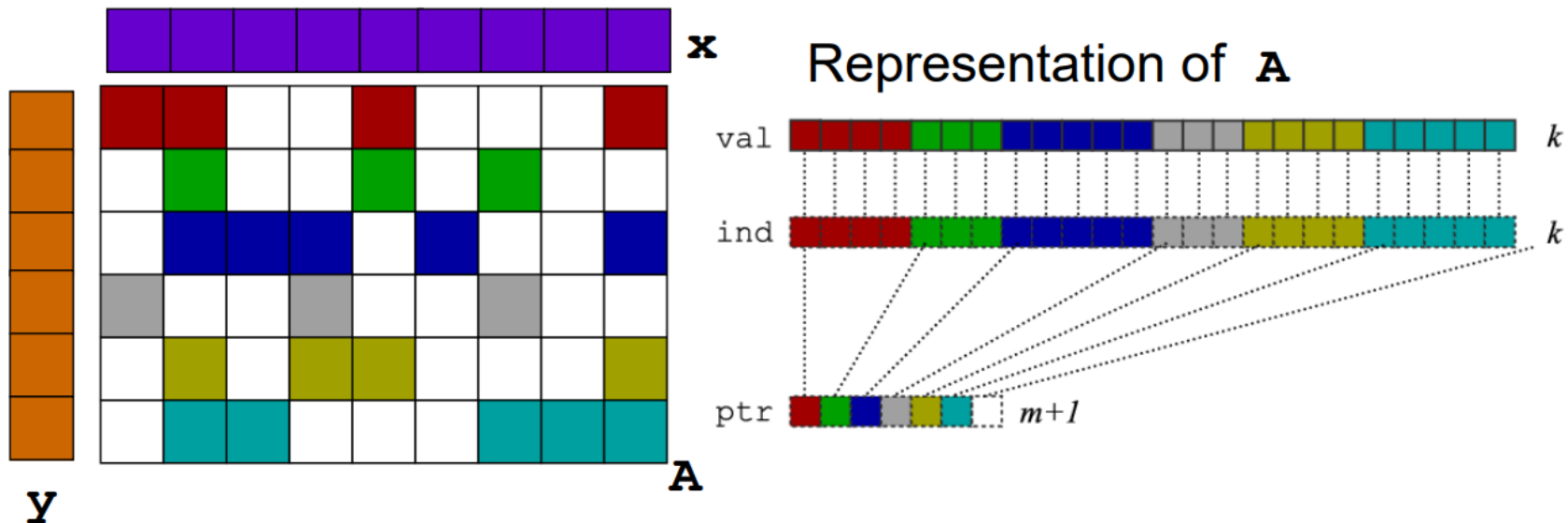
- `row_index = [0,0,1,1,2,2,2,2,2,2,3,3]`
- `col_index = [2,5,2,4,0,1,2,3,6,7,0,3]`
- `value = [a,b,c,d,e,f,g,h,i,j,k,l]`

稀疏矩阵存储格式

		X ₀	X ₁	X ₂	X ₃	X ₄	X ₅	X ₆	X ₇
2	b ₀			a			b		
2	b ₁			c		d			
6	b ₂	e	f	g	h			i	j
2	b ₃	k			l				

- 稀疏行压缩 CSR (Compress Sparse Row) -- 非常常见的存储格式
 - `row_pointer = [0,2,4,10,12]` -- 每行元素开始位置、行数加1
 - `col_index = [2,5,2,4,0,1,2,3,6,7,0,3]`
 - `value = [a,b,c,d,e,f,g,h,i,j,k,l]`
- 稀疏列压缩 CSC (Compress Sparse Column)
 - `col_pointer = [0,2,3,6,8,9,10,11,12]` -- 列数加1
 - `row_index = [2,3,2,0,1,2,2,3,1,0,2,2]`
 - `value = [e,k,f,a,c,g,h,l,d,b,i,j]`

CSR 格式下 SpMV 运算



矩阵向量乘: $y(i) += A(i,j) * x(j)$

```
for (int i = 0; i < m; ++i)
    for (int j = ptr[i]; j < ptr[i + 1]; ++j)
        y[i] += val[j] * x[ind[j]];
```

- x 向量访问不连续
- A 矩阵每行非零元个数不一样

稀疏矩阵计算的挑战

■ 数据局部性问题

- 通过 index 随机访问向量中的某个元素，局部性不好（cache）
- Index 引入更多访存

■ 负载均衡问题

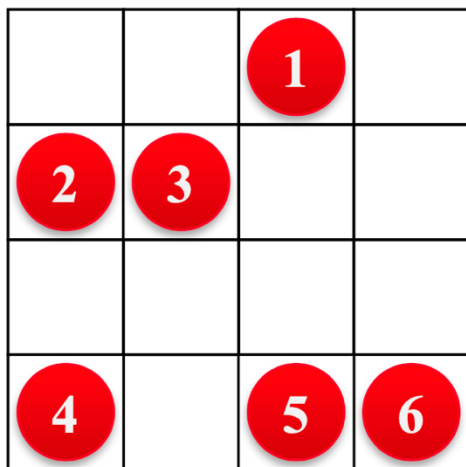
- 稀疏矩阵的不规则性
 - Power-law 矩阵：20% 的行包含了 80% 的元素（甚至更多）
- 任务分配方式对性能有很大影响

■ 写冲突问题

- 多个工作进程可能向结果数组中的同一个变量进行累加
 - 解决方案：原子操作、上锁、分步执行
 - 会引入额外开销

并行 SpMV 算法 – 采用 CSR 格式

- 每个处理器核负责一个行块（多个行）
- 每个行内的乘法操作可以向量化（SIMD）



→ 核心0

→ 核心1

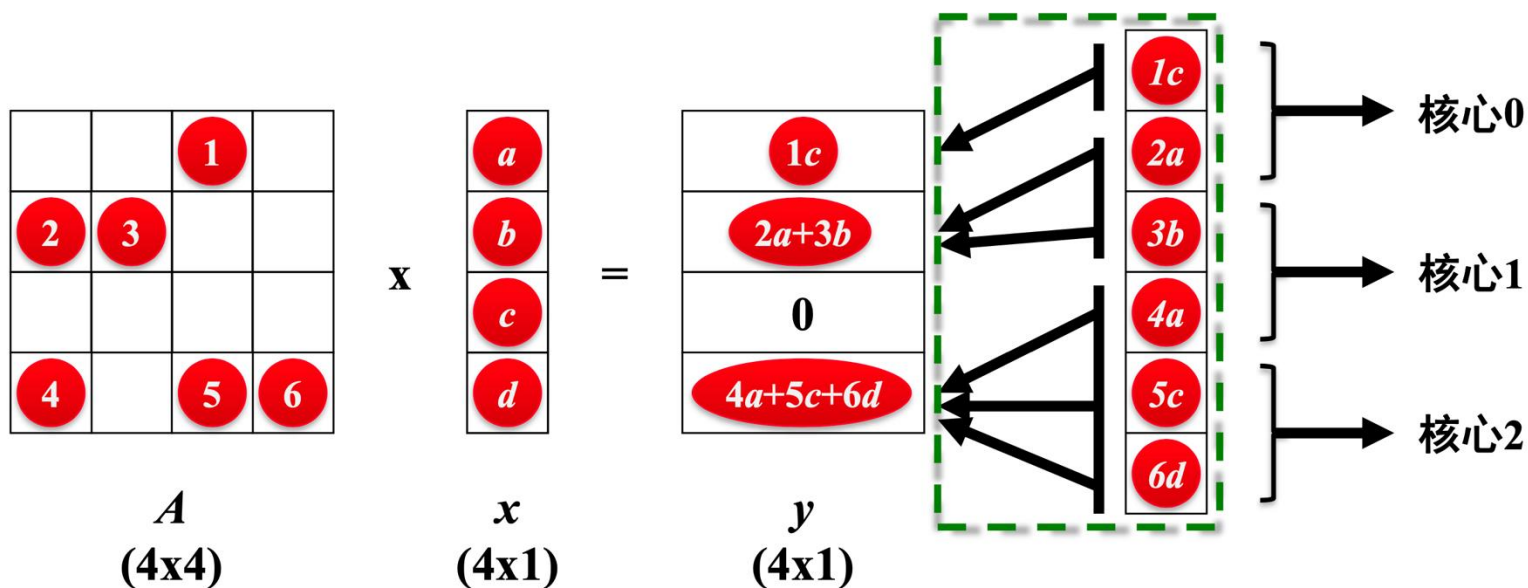
→ 核心2

→ 核心3

- 潜在问题：
 - 有可能负载不均衡
 - 在多核处理器上性能下降

并行 SpMV 算法 – 使用 COO 格式

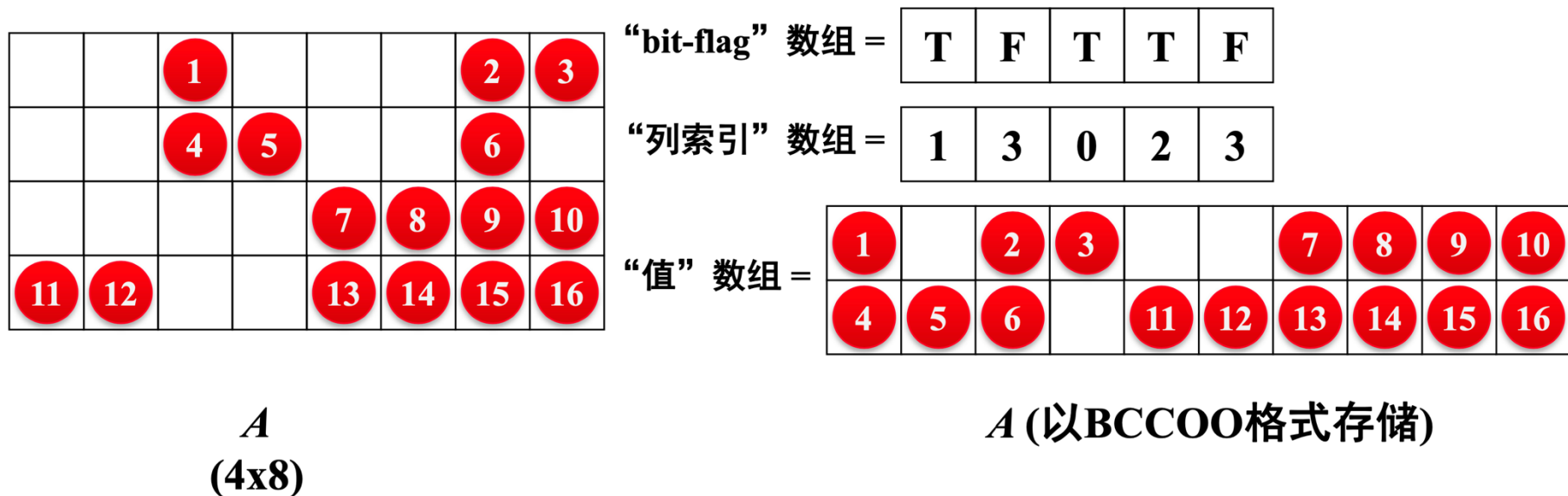
- 全部计算量**均匀分配**所有处理器核心
- 使用**原子加法**或**分段和**进行累加操作



BCCOO 分块存储格式 - yaSpMV

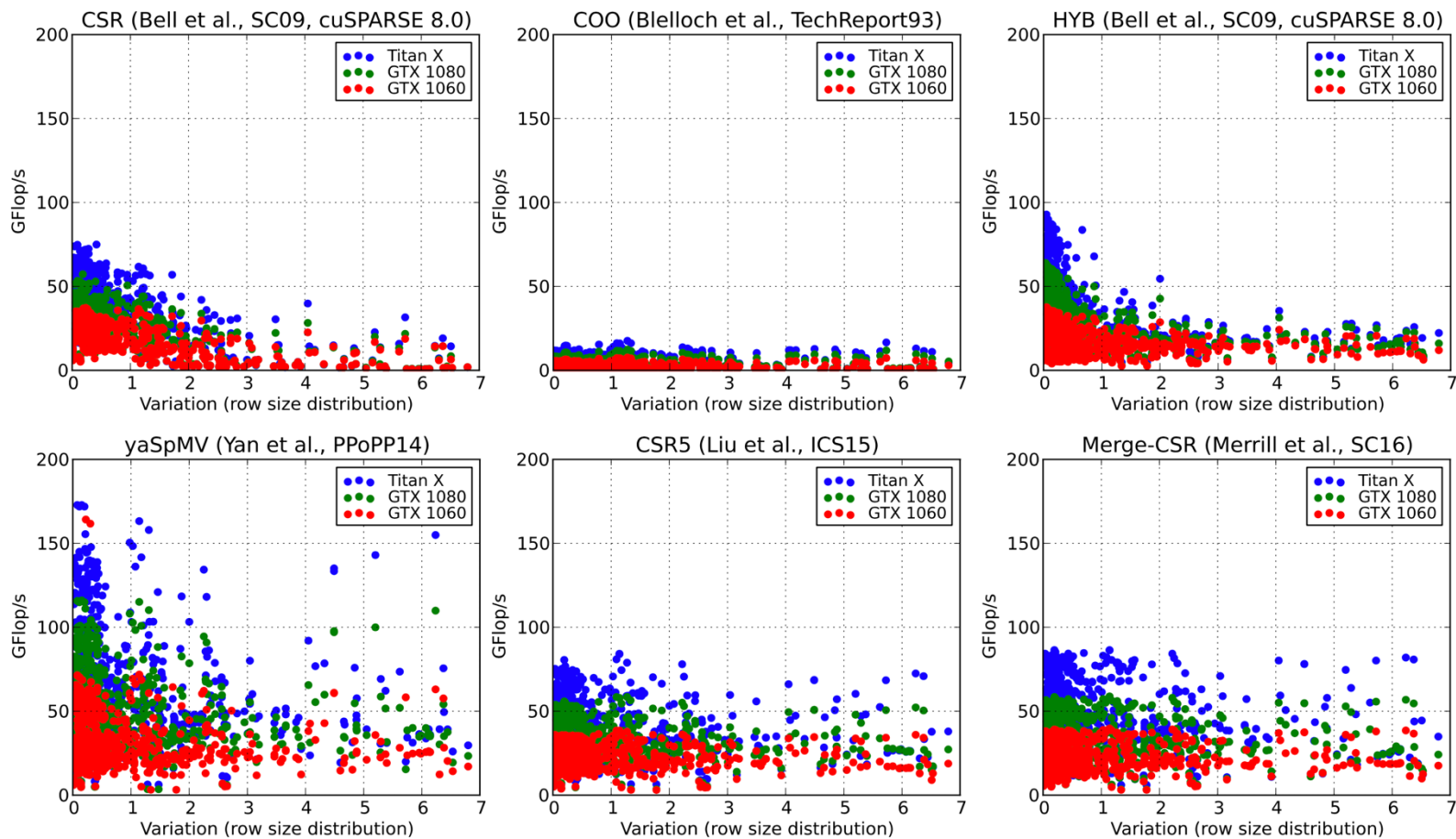
■ yaSpMV:

- 对稀疏矩阵进行**二维分块**，从而增加数据连续性，**减少原子加操作**
- 使用 bit-flag 数组表示**是否是行末**，压缩存储空间
- 使用“**分段和**”改善负载均衡
- 需要根据输入稀疏矩阵**调优块的大小**



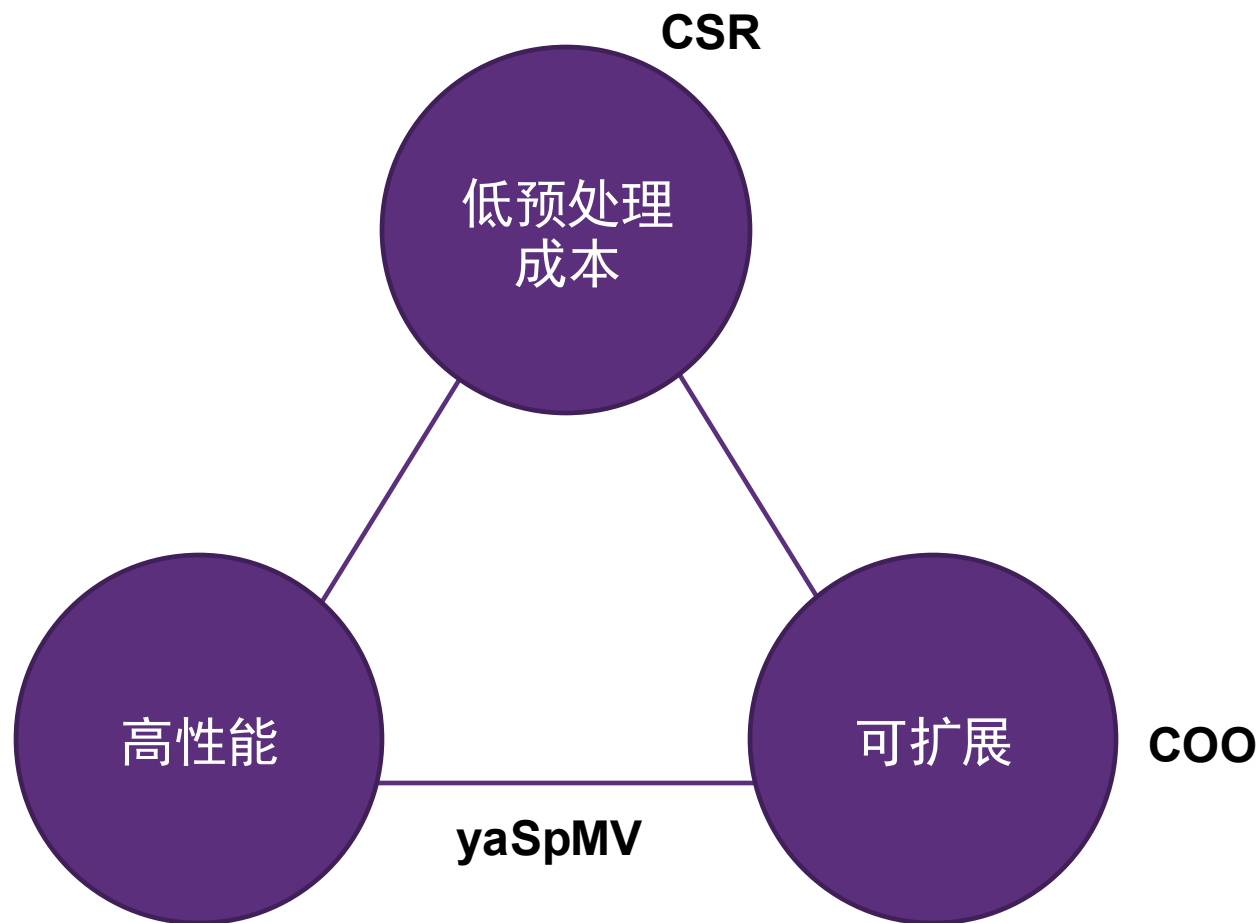
不同并行 SpMV 算法性能对比

- 不同硬件、不同输入数据导致的性能差异极大
- 必需结合实际输入场景讨论稀疏算法的性能



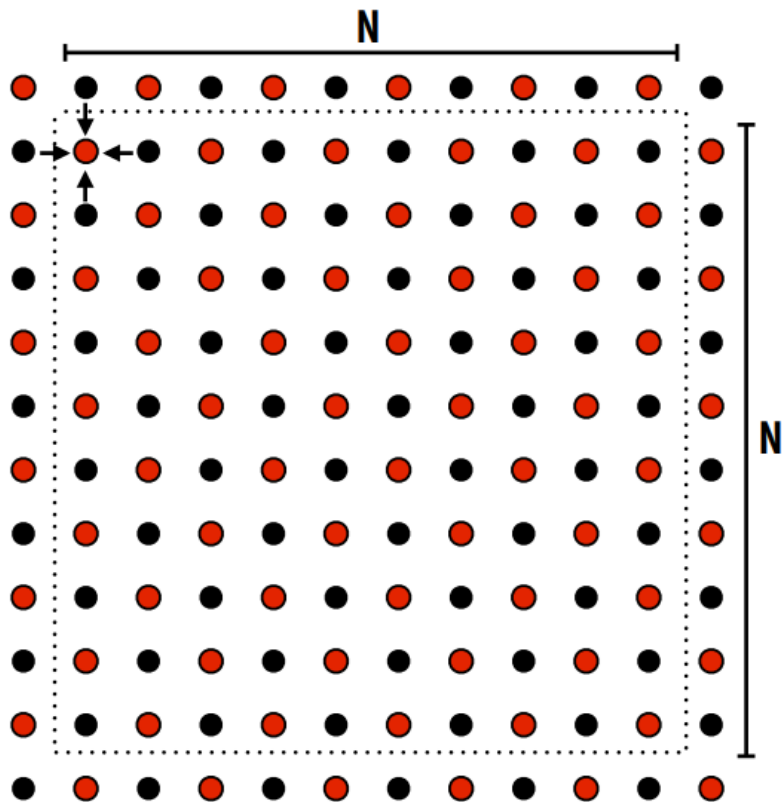
稀疏计算中的“不可能三角”

- 低预处理成本
- 高性能
- 可扩展



消息传递模型

例子：网格求解器

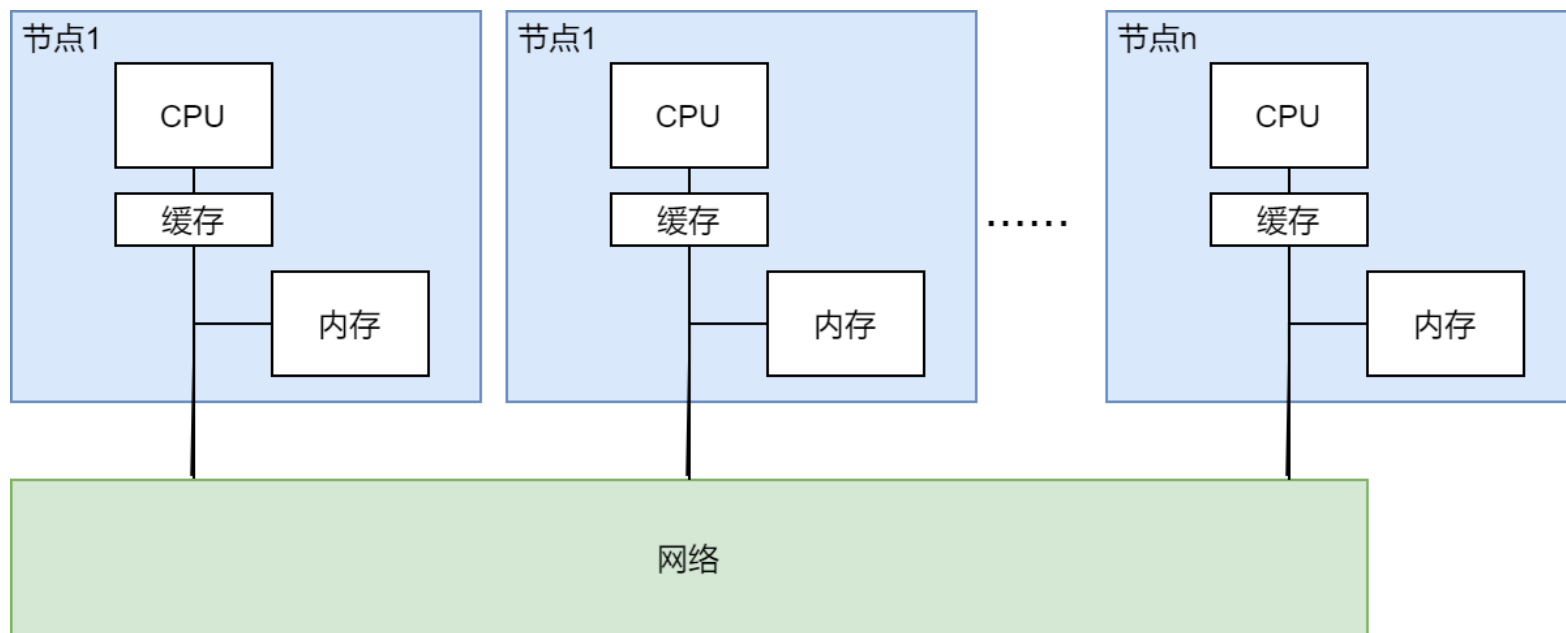


■ 串行算法

```
int N;  
float* A = allocate(N+2, N+2);  
  
void solve(float* A) {  
    bool done = false;  
    float diff = 0;  
    while (!done) {  
        for (all red cells (i, j)) {  
            float prev = A[i, j];  
            A[i, j] = (A[i - 1, j] + A[i, j - 1]  
                    + A[i, j] + A[i + 1, j]  
                    + A[i, j + 1]) / 5;  
            diff += abs(prev - A[i, j]);  
        }  
    }  
}
```

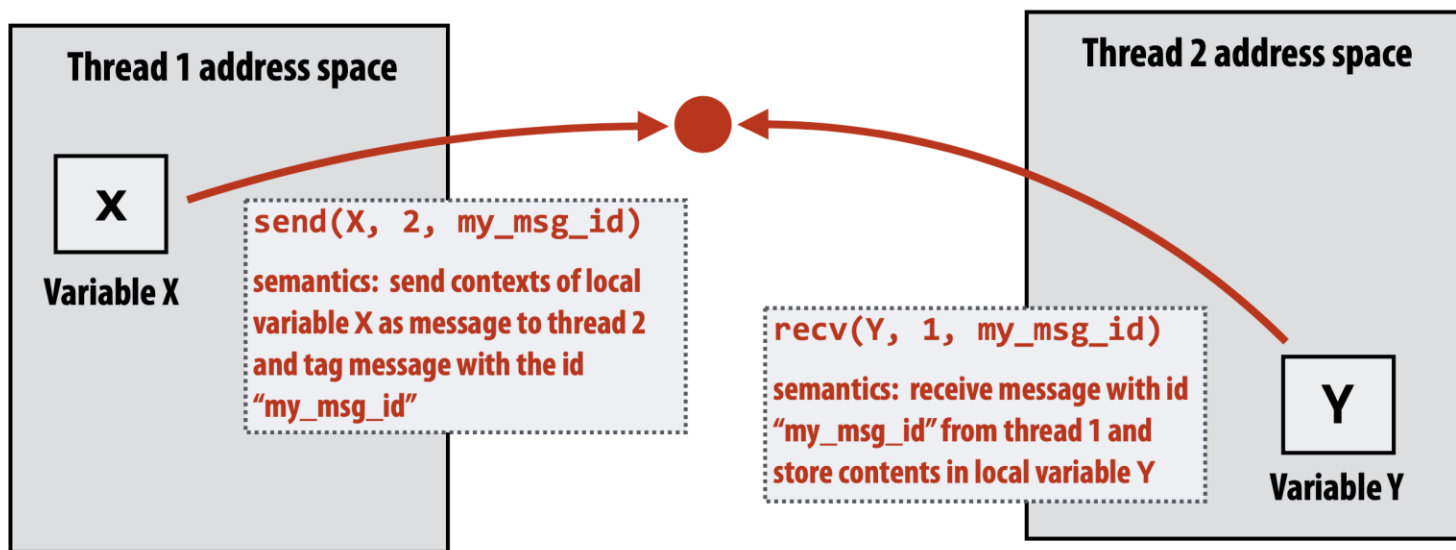
回顾：集群系统

- 典型硬件场景：使用 N 个节点进行并行计算
 - 节点间无共享内存，通过网络连接



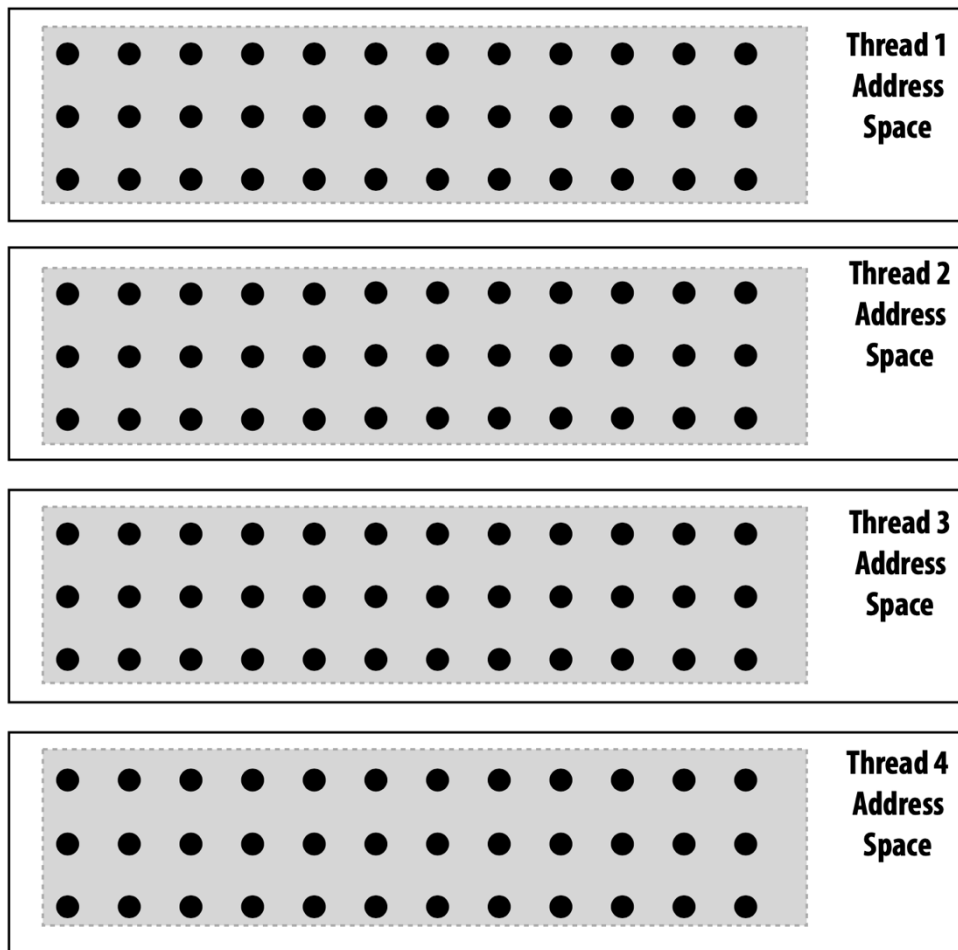
回顾：消息传递模型

- 每个进程（线程）拥有**独立地址空间**
- 进程间通过收发消息来通信
 - **发送消息**（send）：指定接收者、消息大小、消息标识
 - **接收消息**（recv）：指定发送者、消息大小
 - 收发消息是两个进程间唯一的**通信方式**



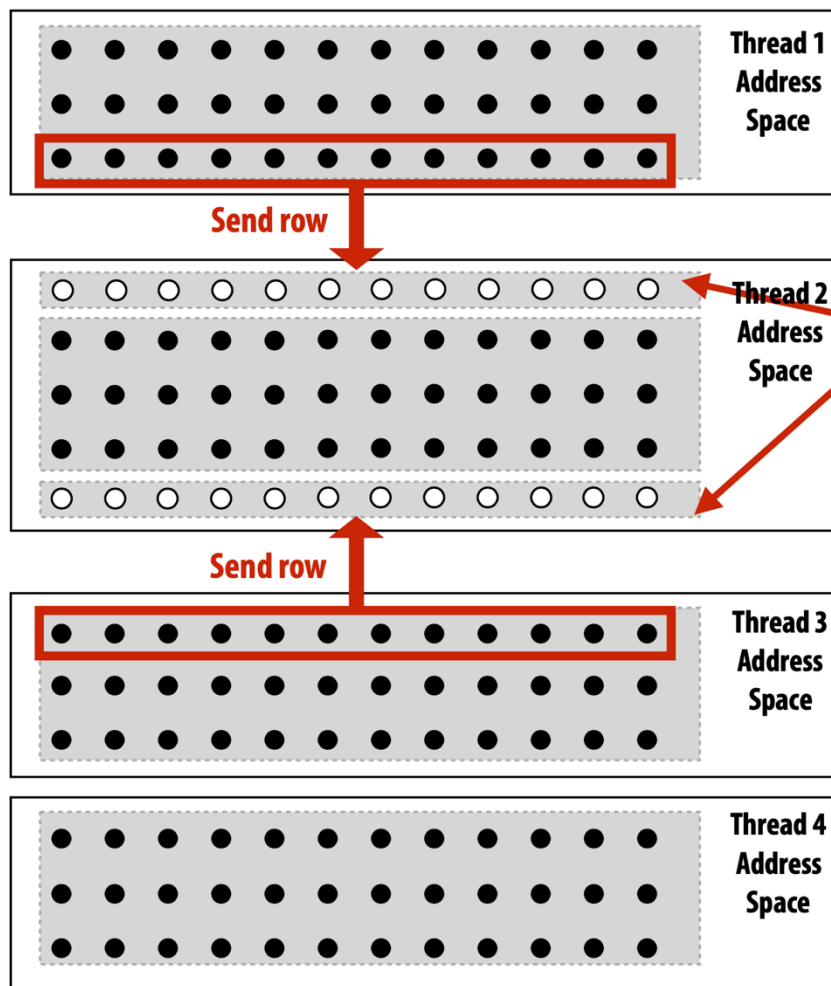
通信示意图

消息传递模型特点：每个进程在私有内存上运行



- 网格数据被分为四份
- 每份放在一个进程的私有地址空间中

数据需要被正确地传递



例子:

- 进程 2 的最上/最下一行依赖于进程 1 和 3 的最下/最上一行
- 进程 1 和 3 需要把对应的行发送给进程 2

“幽灵”网格 (ghost) : 即从其它进程的内存中获取的数据

- “幽灵”网格的数据被其它进程所“拥有”

进程 2 的逻辑如下

```
float* local_data =  
allocate(N+2, rows_per_thread+2);  
int tid = get_thread_id();  
int bytes = sizeof(float) * (N+2);  
// recv ghost row cells  
recv(&local_data[0,0], bytes, tid-1);  
recv(&local_data[row_per_thread+1,0], bytes,  
tid+1);  
// Perform compute
```

基于消息传递：网格求解器程序结构

■ 通信：显式通信

向相邻进程收发“幽灵行”

进行计算

将 diff 汇总到 0 号进程

在进程 0 计算全局 diff，
决定是否要继续计算，
将结果发送给其它进程

```
int N;
int tid = get_thread_id();
int rows_per_thread = N / get_num_threads();
float* localA = allocate(rows_per_thread+2, N+2);
// assume localA is initialized with starting values
// assume MSG_ID_ROW, MSG_ID_DONE, MSG_ID_DIFF are constants used as msg ids
// //////////////////////////////////////
void solve() {
    bool done = false;

    while (!done) {
        float my_diff = 0.0f;
        if (tid != 0)
            send(&localA[1,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
        if (tid != get_num_threads()-1)
            send(&localA[rows_per_thread,0], sizeof(float)*(N+2), tid+1, MSG_ID_ROW);
        if (tid != 0)
            recv(&localA[0,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
        if (tid != get_num_threads()-1)
            recv(&localA[rows_per_thread+1,0], sizeof(float)*(N+2), tid+1,
MSG_ID_ROW);
        for (int i=1; i<rows_per_thread+1; i++) {
            for (int j=1; j<N+1; j++) {
                float prev = localA[i,j];
                localA[i,j] = 0.2 * (localA[i-1,j] + localA[i,j] + localA[i+1,j] +
                    localA[i,j-1] + localA[i,j+1]);
                my_diff += fabs(localA[i,j] - prev);
            }
        }
        if (tid != 0) {
            send(&mydiff, sizeof(float), 0, MSG_ID_DIFF);
            recv(&done, sizeof(bool), 0, MSG_ID_DONE);
        } else {
            float remote_diff;
            for (int i=1; i<get_num_threads()-1; i++) {
                recv(&remote_diff, sizeof(float), i, MSG_ID_DIFF);
                my_diff += remote_diff;
            }
            if (my_diff/(N*N) < TOLERANCE)
                done = true;
            for (int i=1; i<get_num_threads()-1; i++)
                send(&done, sizeof(bool), i, MSG_ID_DONE);
        }
    }
}
```

消息传递模型的例子

- 计算：

- 计算中数组下标都是基于私有内存空间

- 通信：

- 通过收发消息来实现
- 批量传输（Bulk transfer）：每次传输整行

- 同步：

- 通过收发消息来实现

前面实现的网格方程求解器中同步有一个 bug

发生死锁

```
int N;
int tid = get_thread_id();
int rows_per_thread = N / get_num_threads();
float* localA = allocate(rows_per_thread+2, N+2);
// assume localA is initialized with starting values
// assume MSG_ID_ROW, MSG_ID_DONE, MSG_ID_DIFF are constants used as msg ids
//////////
void solve() {
    bool done = false;

    while (!done) {
        float my_diff = 0.0f;
        if (tid != 0)
            send(&localA[1,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
        if (tid != get_num_threads()-1)
            send(&localA[rows_per_thread,0], sizeof(float)*(N+2), tid+1, MSG_ID_ROW);
        if (tid != 0)
            recv(&localA[0,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
        if (tid != get_num_threads()-1)
            recv(&localA[rows_per_thread+1,0], sizeof(float)*(N+2), tid+1,
MSG_ID_ROW);
        for (int i=1; i<rows_per_thread+1; i++) {
            for (int j=1; j<N+1; j++) {
                float prev = localA[i,j];
                localA[i,j] = 0.2 * (localA[i-1,j] + localA[i,j] + localA[i+1,j] +
                    localA[i,j-1] + localA[i,j+1]);
                my_diff += fabs(localA[i,j] - prev);
            }
        }
        if (tid != 0) {
            send(&mydiff, sizeof(float), 0, MSG_ID_DIFF);
            recv(&done, sizeof(bool), 0, MSG_ID_DONE);
        } else {
            float remote_diff;
            for (int i=1; i<get_num_threads()-1; i++) {
                recv(&remote_diff, sizeof(float), i, MSG_ID_DIFF);
                my_diff += remote_diff;
            }
            if (my_diff/(N*N) < TOLERANCE)
                done = true;
            for (int i=1; i<get_num_threads()-1; i++)
                send(&done, sizeof(bool), i, MSG_ID_DONE);
        }
    }
}
```


1888



-

```
int N;
int tid = get_thread_id();
int rows_per_thread = N /
get_num_threads();
float* localA = allocate(rows_per_thread+2,
N+2);
// assume localA is initialized with
starting values
// assume MSG_ID_ROW, MSG_ID_DONE,
MSG_ID_DIFF
// are constants used as msg ids
////////////////////////////////////
void solve() {
    bool done = false;
    while (!done) {
        float my_diff = 0.0f;
        if (tid % 2 == 0) {
            sendDown(); recvDown();
            sendUp(); recvUp();
        } else {
            recvUp(); sendUp();
            recvDown(); sendDown();
        }
    }
}
```

通信优化

通信

- 当我们说到“通信”的时候，并不只包括通过网络的消息传递
- 还包括：
 - 同一块 CPU 上的不同处理器核间的消息传递
 - CPU 和缓存之间的消息传递
 - CPU 和内存之间的消息传递
- 下面讨论通过“流水线”隐藏通信

经典例子：洗衣服

- 洗衣服的步骤：
 - 洗衣服：使用洗衣机，45 分钟
 - 烘干衣服：使用烘干机，60 分钟
 - 叠衣服：使用大学生，15 分钟
- 洗一次衣服的耗时：2 个小时



如何提升洗衣服的吞吐量

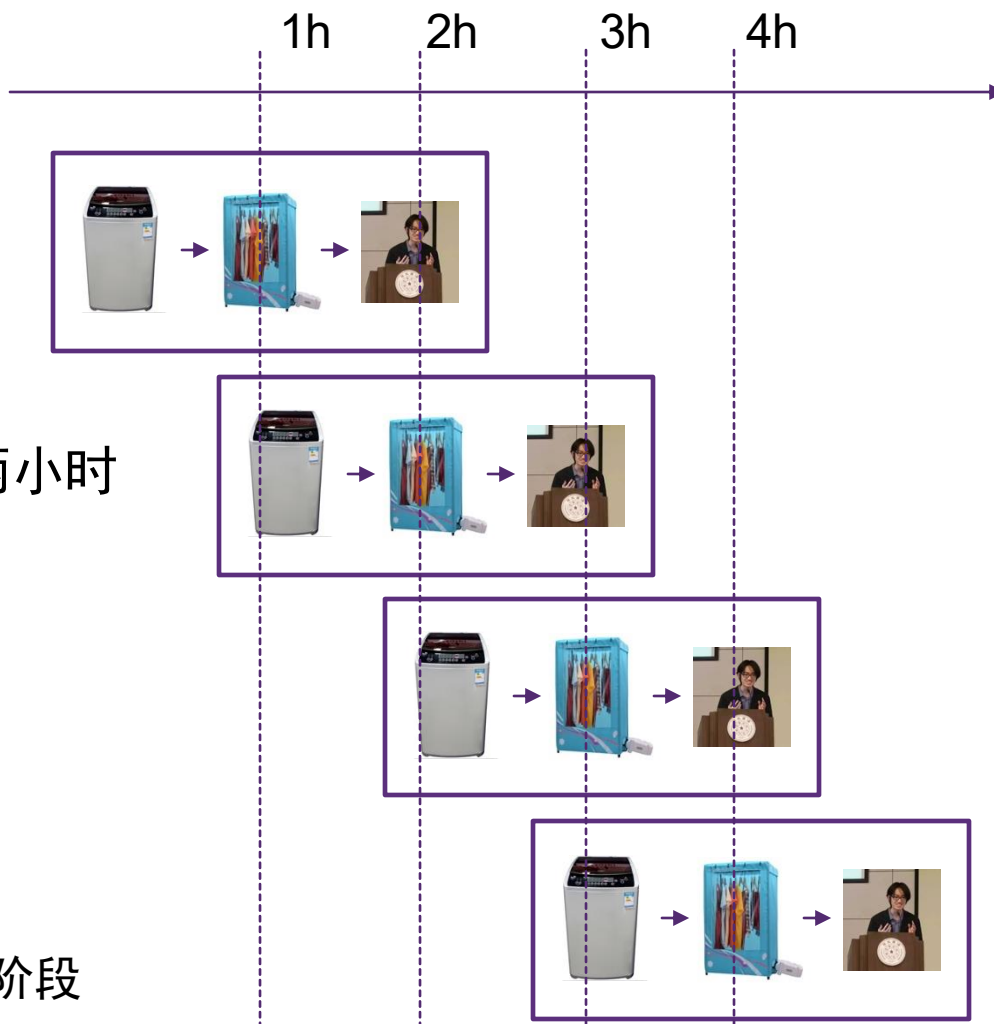
- 目标：当要洗很多批衣服时，提高总吞吐量
- 方案一：增加运行资源
 - 买两份洗衣机，两份烘干机，找两个大学生



- 洗衣服的延迟（latency）：2 小时（没有变化）
- 吞吐量增加两倍：每小时 0.5 份 → 每小时 1 份
- 使用资源增加两倍

流水线加速

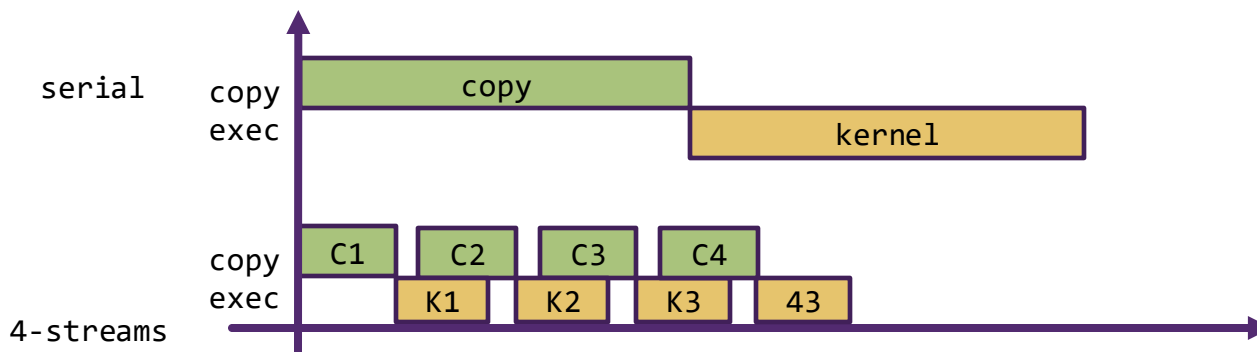
- ✓ **延迟：** 每批衣服需要两小时
- ✓ **吞吐量：** 每小时 1 批
- ✓ **资源：** 不变
- **流水线加速前提：**
 - 任务可以拆分成多个阶段
 - 不同任务之间没有依赖



回顾：GPU 计算与传输流水优化

- 计算和数据传输优化：
 - 对数据进行划分
 - 通过多个 stream 并发执行（任务之间独立）
 - 通过异步实现流水线加速（发射更多的执行单元）

```
size = N * sizeof(float) / nStreams;  
for (i = 0; i < nStreams; i++) {  
    offset = i * N / nStreams;  
    cudaMemcpyAsync(a_d+offset, a_h+offset, size, dir, stream[i]);  
    kernel<<<N / (nThreads * nStreams), nThreads, 0, stream[i]>>>(  
a_d + offset);  
}
```



计算与通信重叠

- 使用异步 send / recv
- 在通信的同时进行其它计算

异步启动通信
计算

进行依赖通信的计算

```
int N;
int tid = get_thread_id();
int rows_per_thread = N / get_num_threads();
float* localA = allocate(rows_per_thread+2,
N+2);
// assume localA is initialized with starting
values
// assume MSG_ID_ROW, MSG_ID_DONE, MSG_ID_DIFF
are constants used as msg ids
////////////////////////////////////
void solve() {
    bool done = false;
    while (!done) {
        float my_diff = 0.0f;
        sendUpAsync(); hdl1 = recvUpAsync();
        sendDownAsync(); hdl2 = recvDownAsync();
        for (int i = 1; i + 1 < rows_per_thread; ++i)
            computeRow(i);
        wait(hdl1);
        wait(hdl2);
        computeRow(0);
        computeRow(rows_per_thread - 1);
        ...
    }
}
```


通信计算比

$$\text{通信计算比} = \frac{\text{通信量 (如字节数)}}{\text{计算量 (如指令数)}}$$

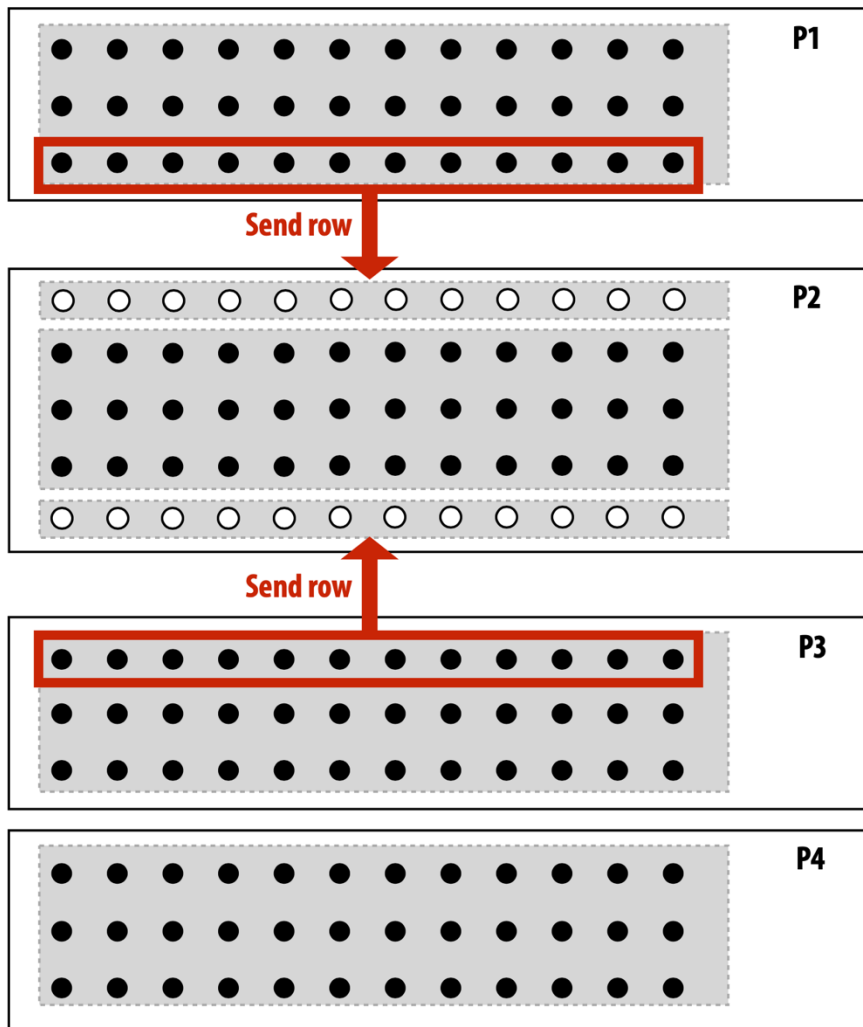
- 如果分母是计算执行时间，这个比值给出了程序所需的通信带宽
- “算术密度” (arithmetic intensity) = 1 / 通信计算比
 - 更直观指标
 - 越高越好
- 现代计算机体系结构中，计算性能远比通信高
 - 需要更高的算术密度（更低的通信计算比）来充分发挥硬件资源
 - 回顾：GPU 矩阵乘优化例子（更高的计算访存比）

通信开销

通信来源

- 原生通信 (inherent communication)
 - 假设无限的缓存、最小粒度的传递
 - 给定并行算法和划分方案, 必需发生的通信
- 额外通信 (artifactual communication)
 - 所有其它通信
 - 和实现细节有关

原生通信

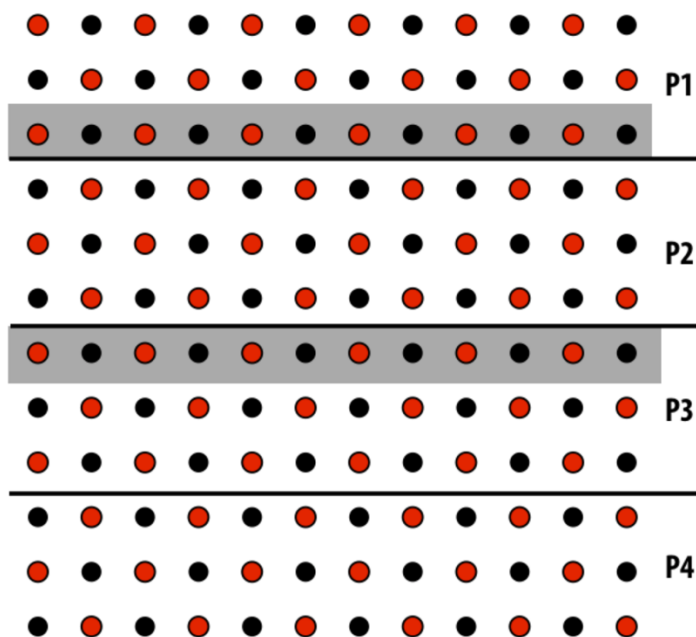


- 在并行程序中**必然发生**的通信
- 是算法的**基础构成部分**
- 例子：
 - 网格方程求解中**传递幽灵行**

减少原生通信

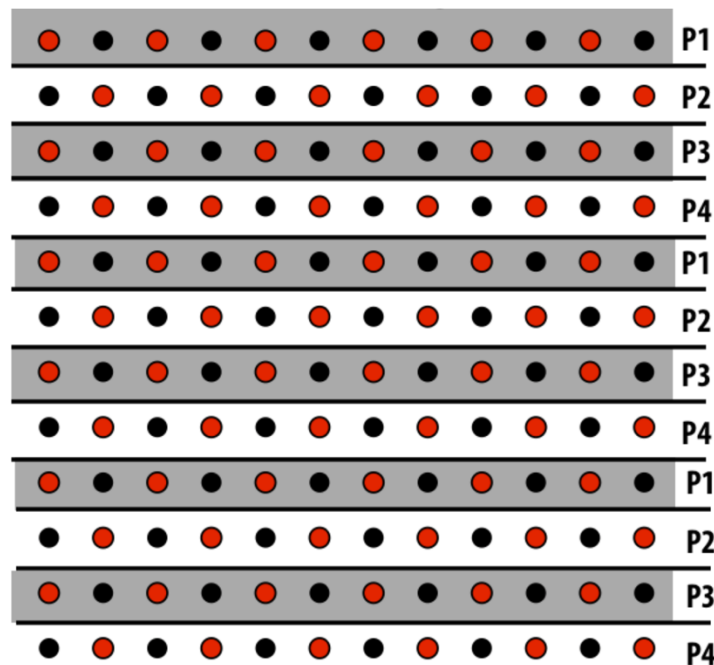
- 好的任务分配方案有助于减少原生通信
 - 改进算术密度

1D blocked assignment: N x N grid



$$\frac{\text{elements computed (per processor)} \approx N^2/P}{\text{elements communicated (per processor)} \approx 2N} \propto N/P$$

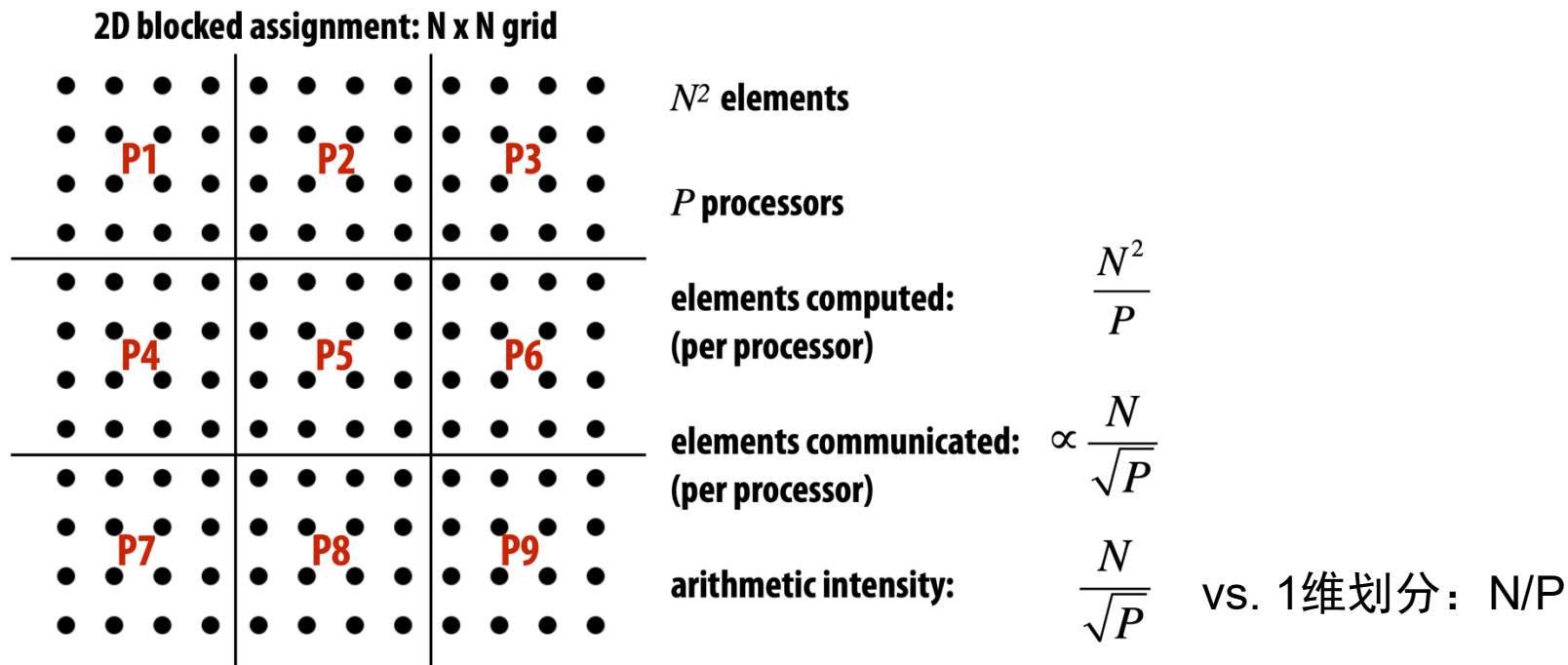
1D interleaved assignment: N x N grid



$$\frac{\text{elements computed}}{\text{elements communicated}} = 1/2$$

算术密度低

减少原生通信



算数密度高

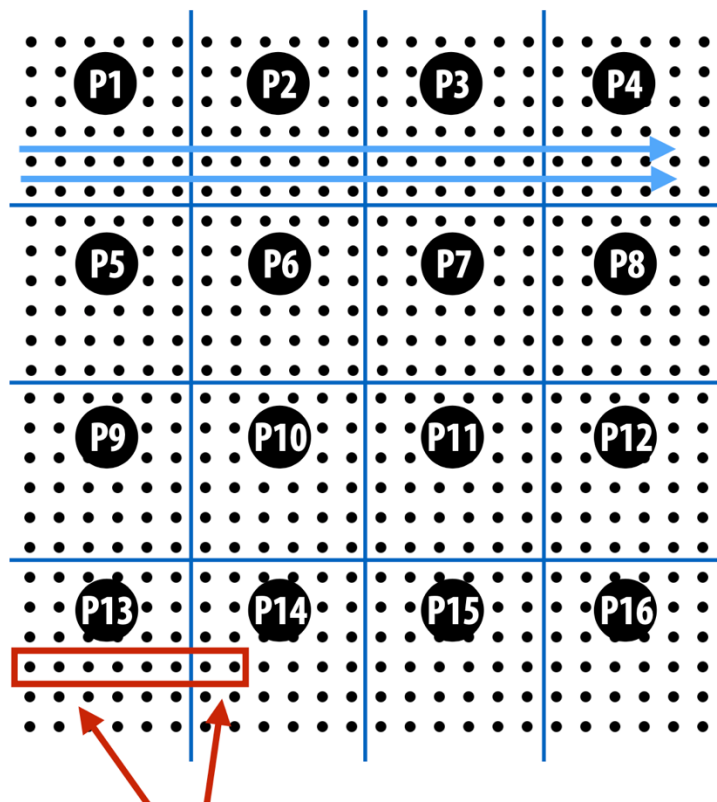
- 2 维划分：
 - 每个处理器比 1 维划分有**更小的通信量** ($N/\sqrt{P}$ vs. $2N$)
 - P 个处理器总通信量和 P 呈弱线性相关 ($N\sqrt{P}$)
 - 发挥了二维空间的局部性

额外通信例子

- 最小数据传输粒度
 - 导致系统比需要传输更多的数据
 - 例子1: cache line 大小为 64 字节, 但是只需要访问 4 字节的数据
 - 引入了额外的 16 倍通信
 - 例子2: GPU 全局内存访问粒度 sector (32Byte)
- 不必要的通信
 - 写 16 个 4 字节数据, 但是跨了 cache line, 所以多了一倍的通信
- 缓冲区容量有限
 - 受到 cache 大小限制, 需要多次传输重复数据

减少额外通信：数据布局

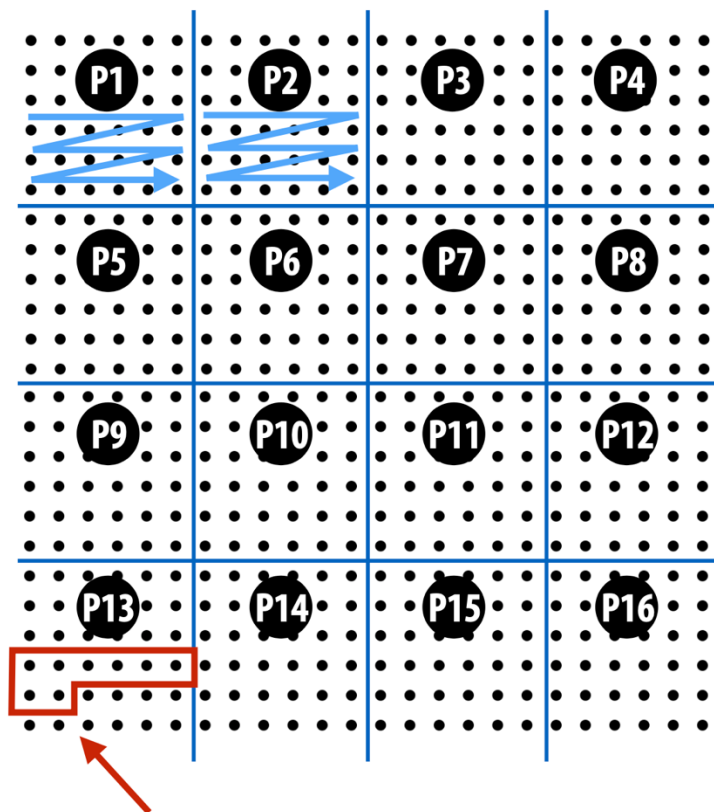
2D, row-major array layout



按照整行存储 → 连续地址存放不同块的数据

假设 cache line: 4个元素

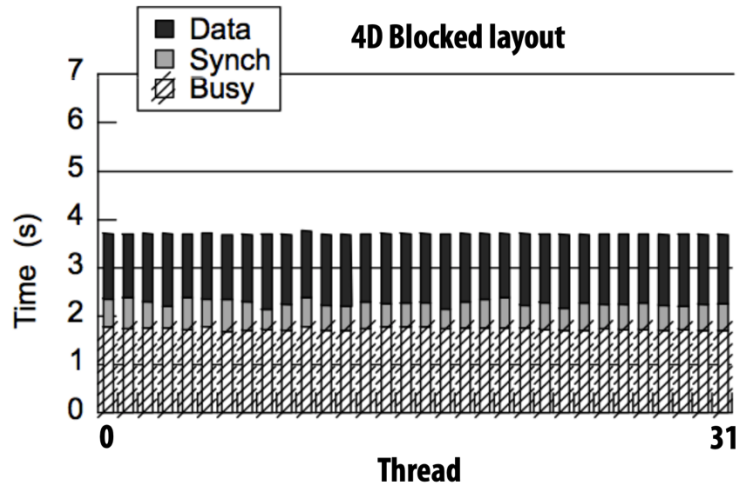
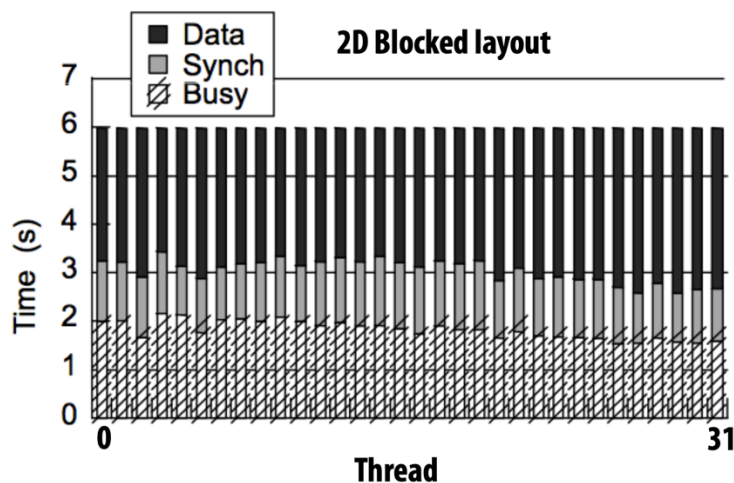
4D array layout (block-major)



按照块连续存储 → 连续地址存放连续块的数据

网格方程求解器的例子：运行时间分解

Execution on 32-processor SGI Origin 2000 (1026 x 1026 grids)



■ 观察：

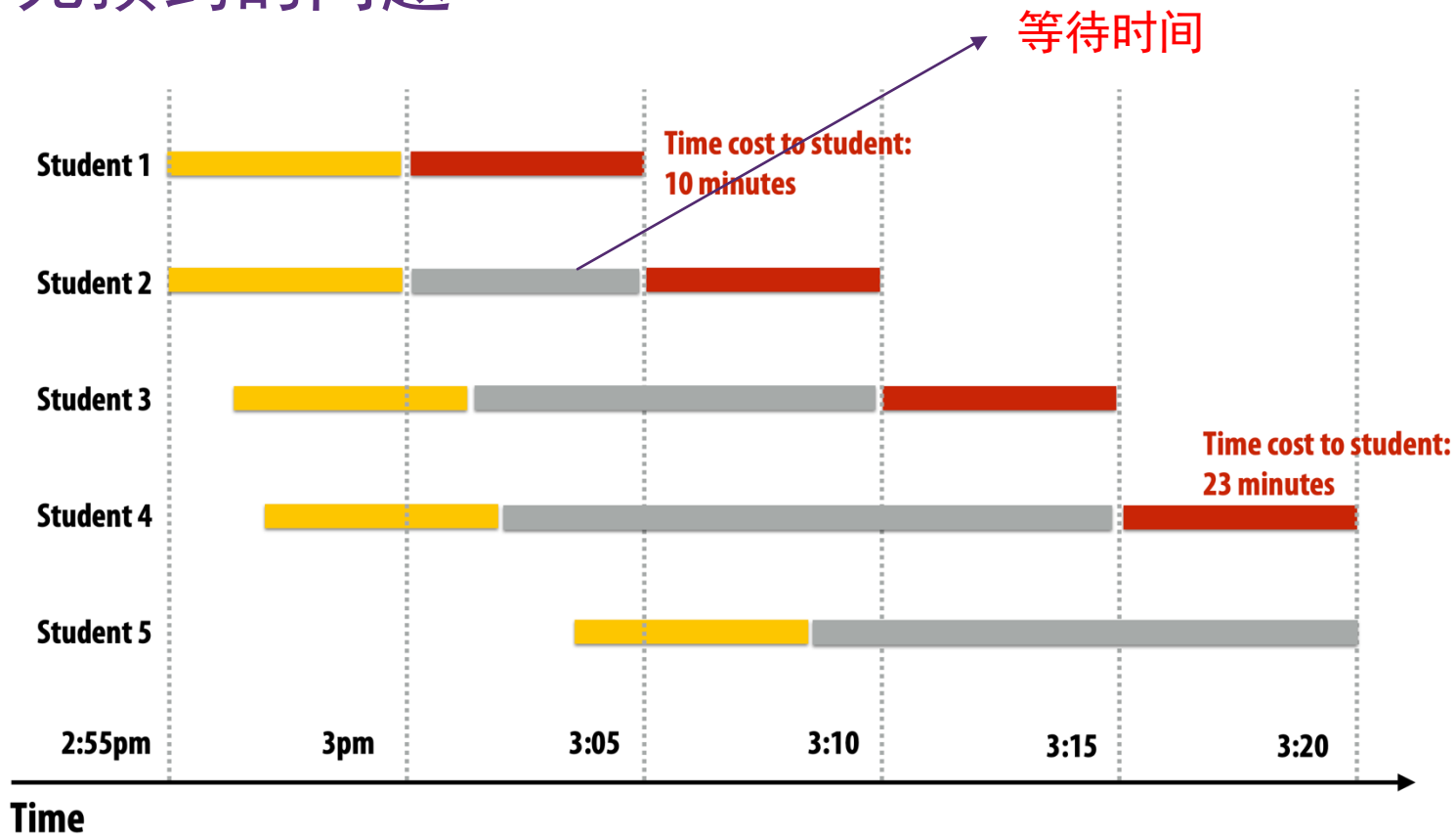
- 4D 划分减少了数据传输时间（表现在端到端时间的减少）

通信拥塞

例子：office hour（无预约）

- 操作：任课老师回答一位同学的问题
- 执行资源：任课老师
- 执行步骤
 - 同学从紫荆公寓骑车到办公楼：5 分钟
 - 同学排队（如果需要排队的话）
 - 同学和老师交流，获得想要的答案：5 分钟

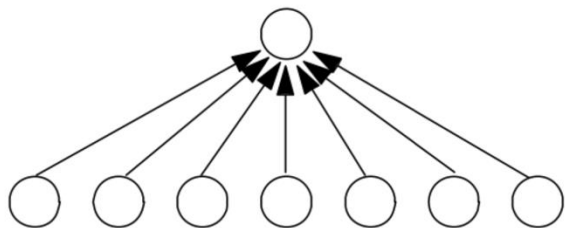
无预约的问题



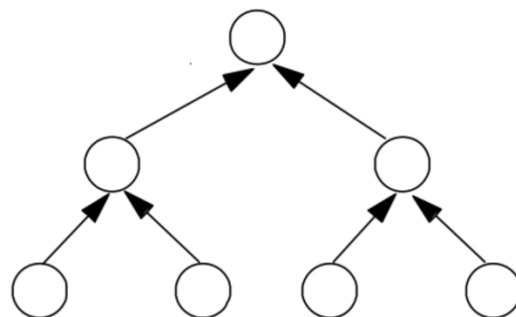
- 共享资源拥塞，总操作时间变长
- 每个同学消耗的总时间变长

拥塞 - Contention

- 一个硬件资源可以以某个吞吐量执行某种操作
 - 吞吐量：每秒处理事务的数量
 - 例子：内存带宽、网络链路、数据库服务器等
- **拥塞**：当短时间内超过吞吐量的请求被发送到该资源，就发生了拥塞
 - 该资源被称作热点（hotspot）
- 例子：共享变量累加

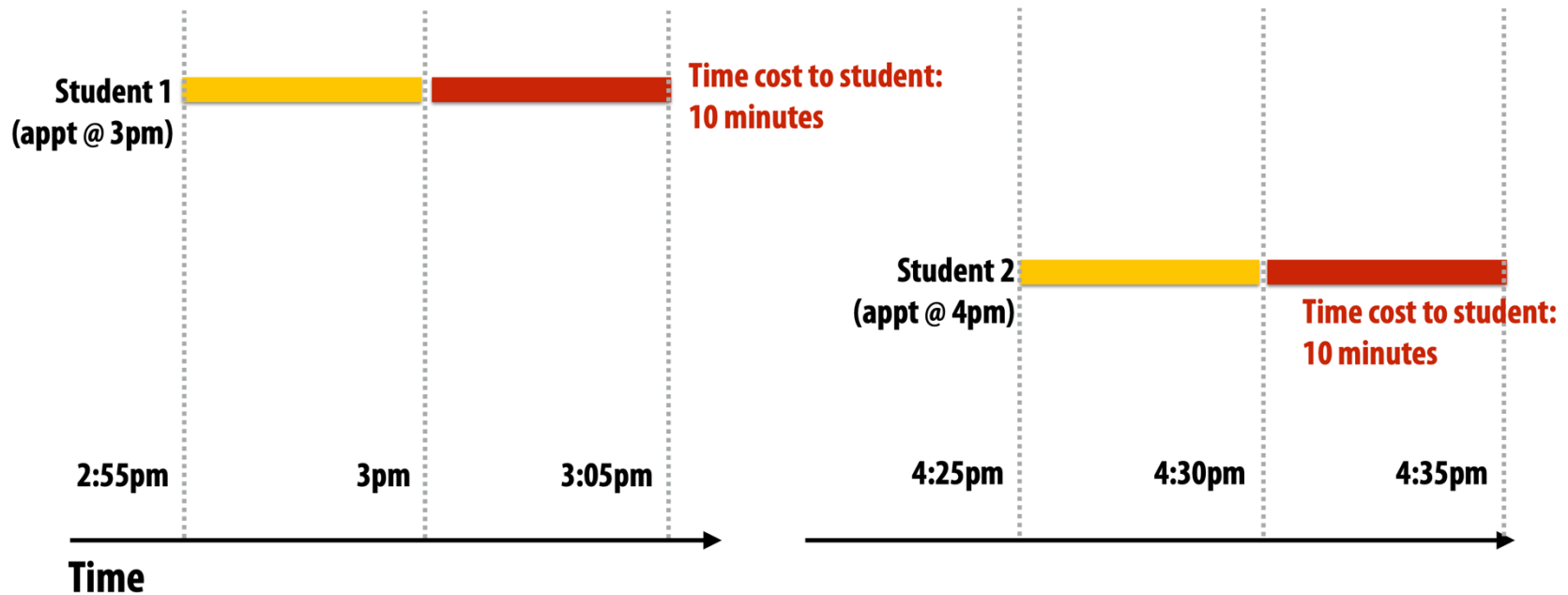


扁平方式：很可能发生拥塞，但不拥塞时**延迟较低**



树状结构：减少拥塞，但在不拥塞时延迟更高

避免拥塞：通过预约



- 预约可以有效减少拥塞
- 预约代价：
 - 额外通信
 - 需要更复杂的时间管理
 - 预约冲突的处理

减少拥塞：分布式队列

- 避免所有工作线程都在**同一个队列上同步**

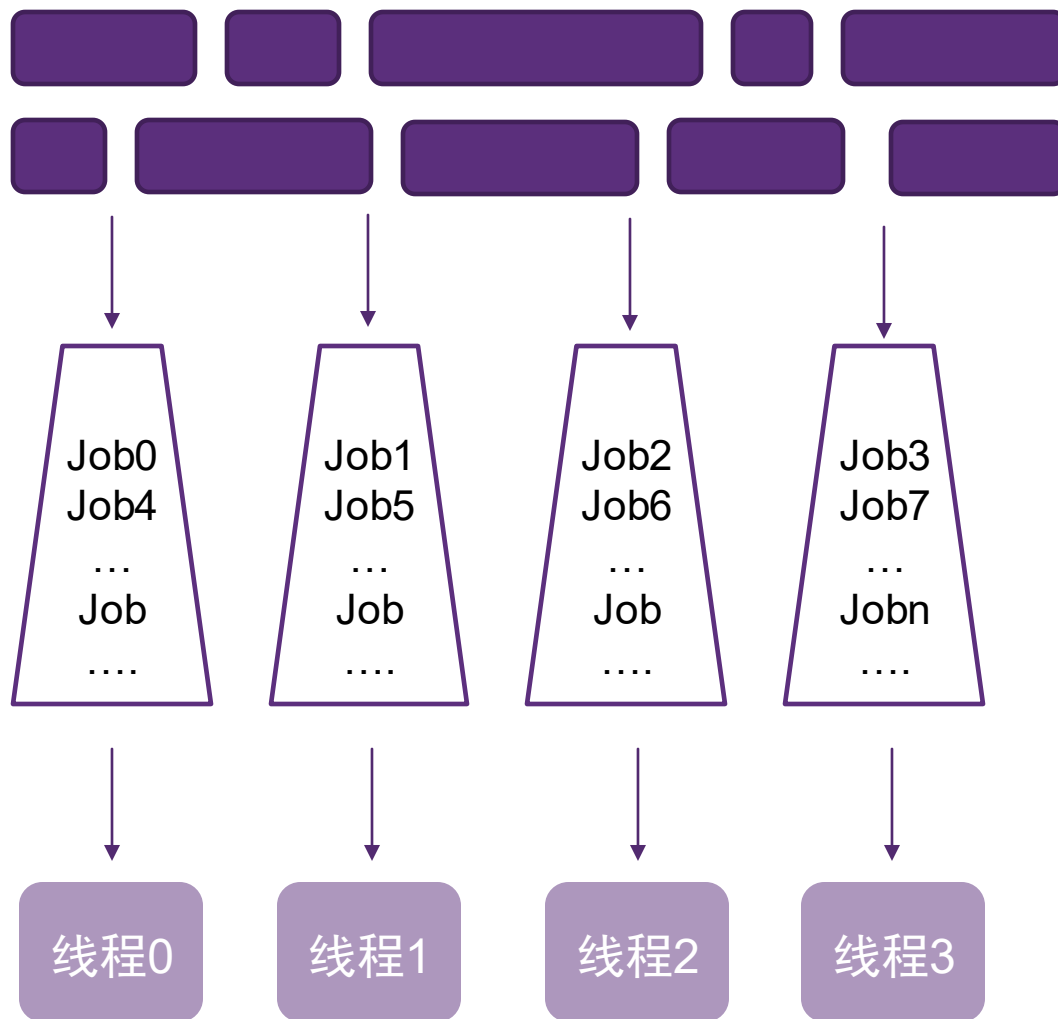
- 子任务：

- 一组任务队列：

- 队列不会拥塞

- 工作线程：

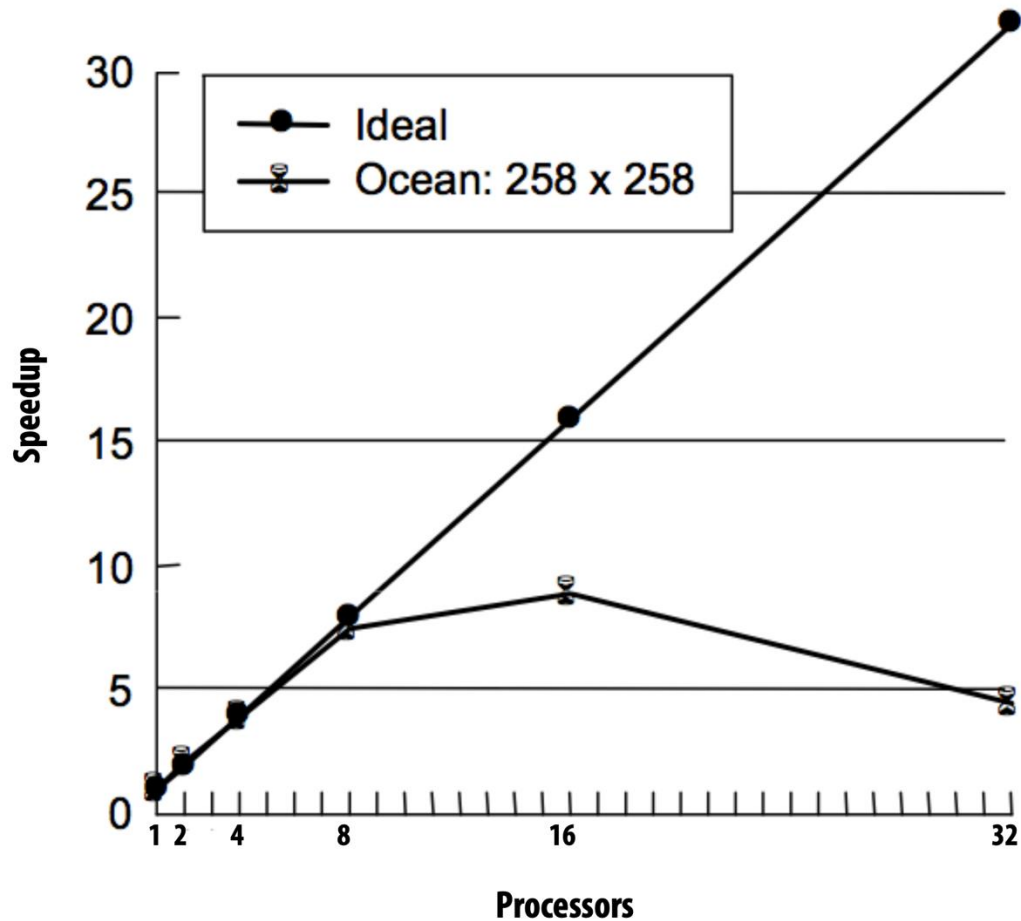
- 往**自己的**队列里推任务
- 从**自己的**队列里取任务
- 当自己的队列空了
 - 从其它队列“偷”任务 (work stealing)



可扩展性讨论

网格方程求解器的加速比

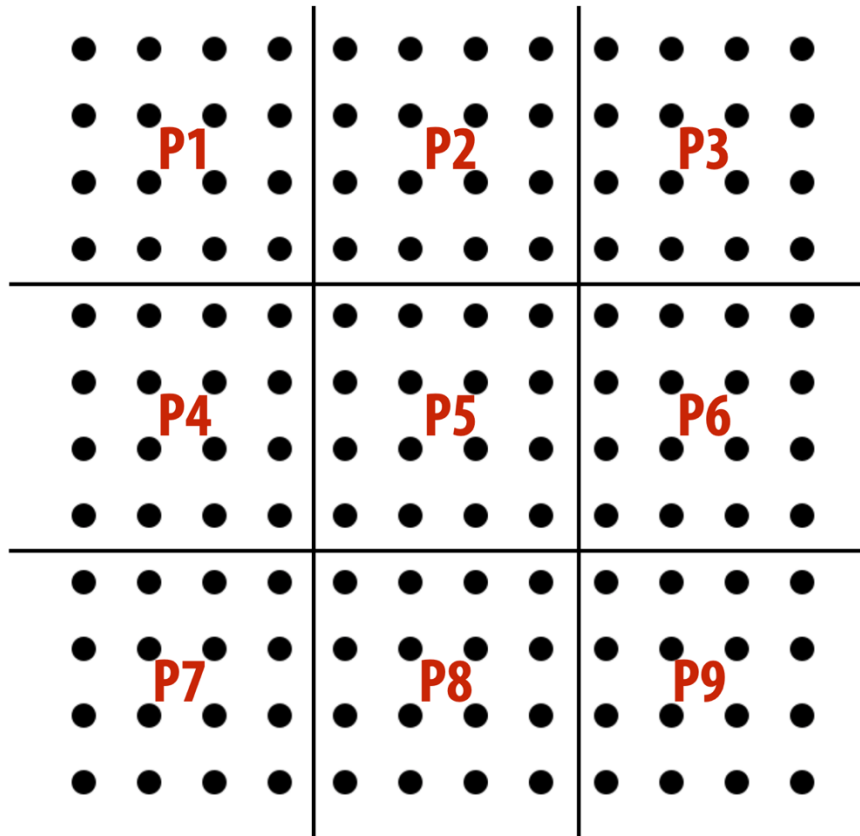
- 使用 32 个 SGI Origin 2000 处理器 (258 x 258 网格)



加速比在 16 个处理器后明显下降

网格方程求解器算数密度

2D blocked assignment: $N \times N$ grid



N^2 elements

P processors

elements computed:
(per processor)

$$\frac{N^2}{P}$$

elements communicated:
(per processor)

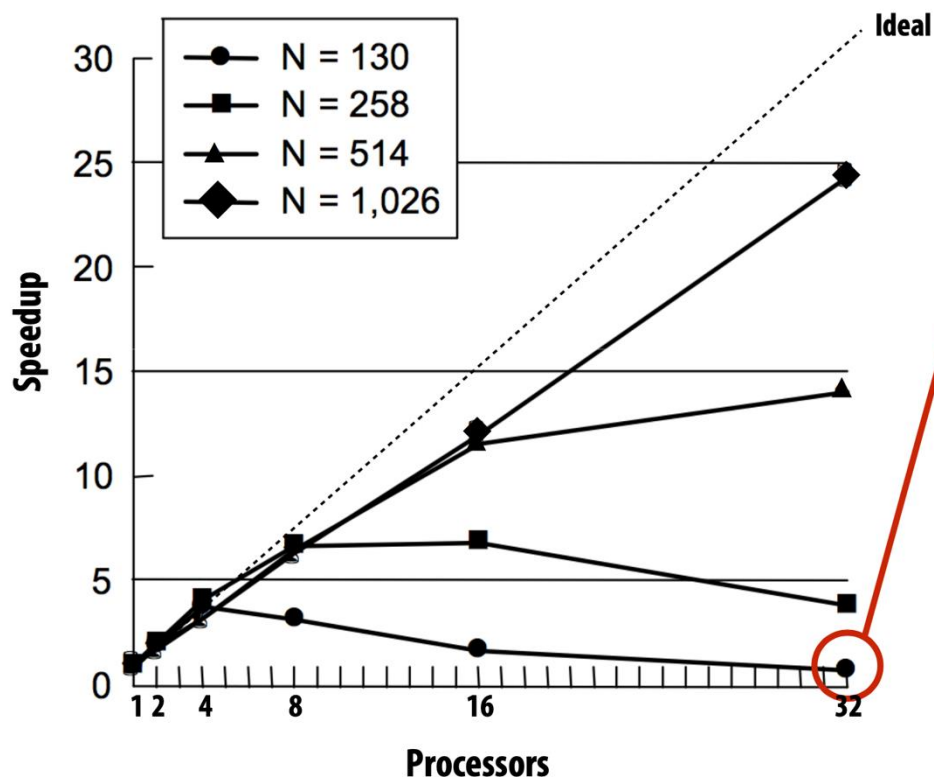
$$\propto \frac{N}{\sqrt{P}}$$

arithmetic intensity:

$$\frac{N}{\sqrt{P}}$$

更小的 N 或更大的 P 都会导致算术密度降低！

固定问题规模的陷阱-1

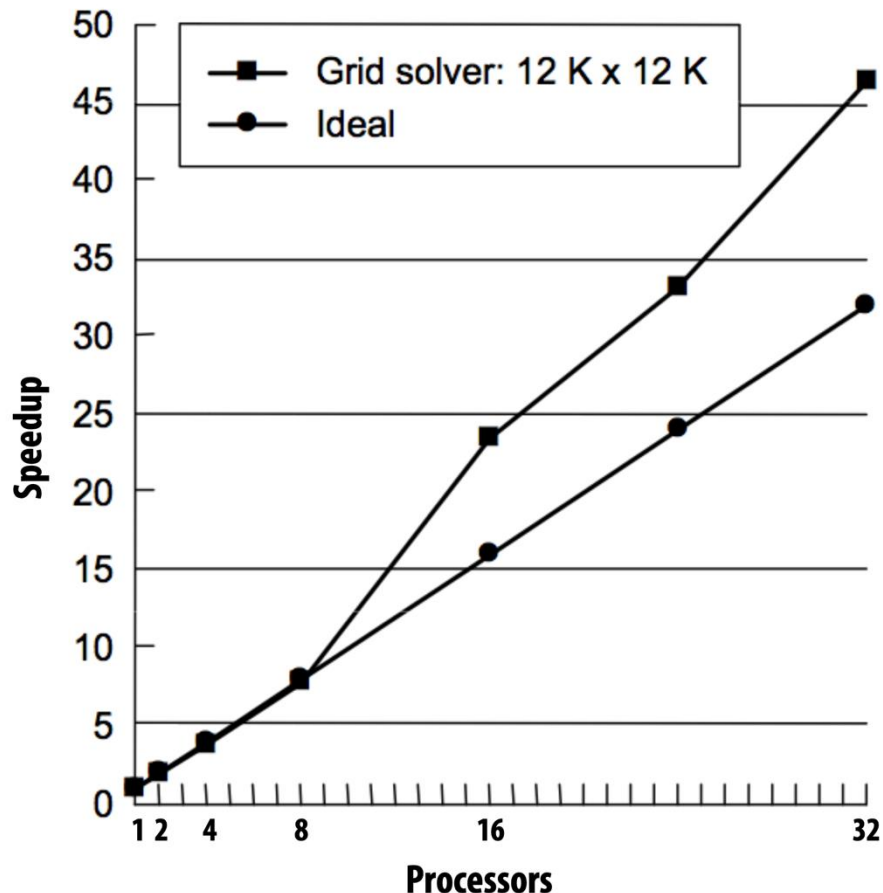


- ✓ N = 130 在 32 处理器上，没有好处，性能下降！
- ✓ 问题规模对于机器来说太小，导致极大的通信-计算比
- ✓ 对于小问题，扩展到大规模机器上已经不太必要（单核已经够快了）

32 核上处理 258x258 的网格：每个进程 2K 个网格

32 核上处理 1Kx1K 的网格：每个进程 32K 个网格

固定问题规模的陷阱-2



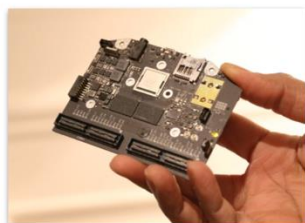
- **超线性加速比！**
 - 随着处理器数量增加，分配给每个处理器的网格能够**放入缓存**
- 反例：当问题规模太大，内存中**存不下工作集**，数据被交换到硬盘，性能急剧下降
- 在规模更大、内存更大的机器上做并行程序开发十分吸引人

理解可扩展性

- 问题规模和并行计算机规模之间有**复杂关系**（N 和 P）
 - 影响负载均衡、额外开销、算术密度、数据局部性
 - 和应用程序特性相关
- 以**固定问题规模**来评价并行性能是有问题的
 - **问题太小**
 - 并行的额外开销掩盖了并行的收益
 - 问题规模可能只适用于小机器，而不适用于大机器
 - **问题太大**
 - 机器的内存可能无法容纳所需的数据集
 - 导致无法运行或者交换到硬盘上、无法有效评估
- **问题规模需要随机器大小而改变**
 - 一个更大的机器应该解决更大的问题规模

系统设计师：对可扩展性的考虑

- 一个常见问题：这个系统有很好的扩展性吗？
- 从软化的角度：
 - 向上扩展（scale up）：当核数增加时，性能会增加吗？
 - 高端用户的关心
 - 向下扩展（scale down）：当核数减少时，性能降低多少？
 - 低端用户的关心
- 从硬件的考虑：
 - 相同的体系结构，适用不同的用户
 - 例子：GPU 架构
 - 相同的架构，不同 SM 数量，不同性能，不同价格



Tegra X1: 2 SMM cores
(mobile SoC)



GTX 950: 6 SMM cores
(90 watts)



GTX 980: 16 SMM cores
(165 watts)



Titan X: 24 SMM cores
(250 watts)

当考虑可扩展性时应该思考的问题

- 问题应该按照**什么方式来扩展**？

- 固定问题规模
- 固定执行时间
- 固定内存使用量

- **问题规模如何衡量**

- 问题规模通常由多个变量决定
- 例子 → 网格方程求解器
 - 输入网格大小
 - 误差精度要求
 - 时间步的大小
 - 待模拟的时间

固定问题规模可扩展问题

- 关注点：适用并行计算机更快的解决固定问题规模
 - 加速比 =
$$\frac{\text{使用 1 个处理器的时间}}{\text{使用 p 个处理器的时间}}$$
- 考虑固定问题规模的陷阱
 - 问题规模太小不适合大机器
 - 问题规模太大不适用小机器
- 应用例子：
 - 求解一个更大规模的线性方程组

固定时间可扩展问题

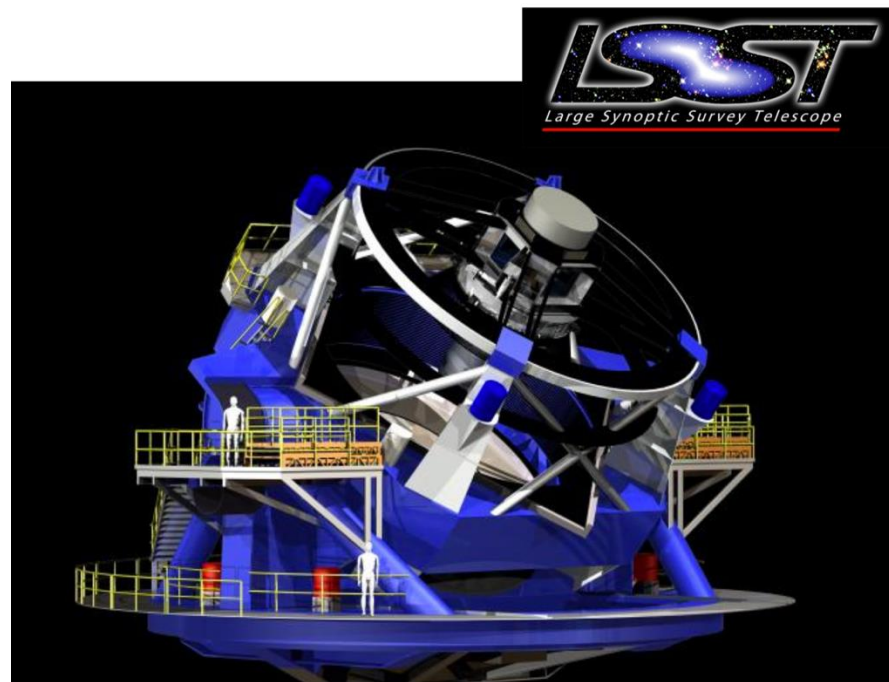
- 关注点：在**固定时间**内的完成更多的工作量
 - 工作时间不变，问题规模变大
 - 加速比 =
$$\frac{\text{使用 } p \text{ 个处理器完成的工作量}}{\text{使用 } 1 \text{ 个处理器完成的工作量}}$$
- 如何定义完成“工作量”？
 - **挑战**：完成的工作和输入的问题规模不是线性函数（矩阵乘： $O(N^3)$ 的计算对 $O(N^2)$ 的输入）
 - **一个解决方法**：定义的工作量指用单处理器完成同样计算**所需时间**
 - 理想情况下，工作量的定义方法是
 - 简单易懂
 - 与串行执行时间线性相关（理想的加速比和 P 成比例）

固定时间可扩展的例子

- 大规模天空**扫描望远镜**
 - 2019 年建成
 - 获得高分辨率天空扫描图像
 - 每 15 秒扫一张 30 亿像素图像
 - 年复一年地运行

对图片进行分析
找到“可能有趣”的图片

“有兴趣”的
下游研究人员



LSST will be located on top of Cerro Pachón Mountain, Chile

- ✓ 更强的算力运行使用**更复杂的检测算法**
 - ✓ 检测更精准
 - ✓ 可以检测更多的兴趣点

更多固定时间可扩展的例子

■ 高频交易

- 在相同时间内跑更复杂模型
- 1ms、1min

■ 现代网站

- 在 x 毫秒延迟内生成更复杂的网页

■ 实时图像分析

- 自动驾驶车：在有限时间（5ms）内实现更高质量的物体检测算法

固定内存可扩展问题

- 关注点：在不超过**内存限制**的前提下运行更大的任务
 - 每处理器的**内存使用是固定的**
 - 问题规模和机器规模并不固定
- **加速比** =
$$\frac{\text{使用 } p \text{ 个处理器完成工作量} \times \text{使用 } 1 \text{ 个处理器执行时间}}{\text{使用 } p \text{ 个处理器执行时间} \times \text{使用 } 1 \text{ 个处理器完成工作量}}$$
$$= \frac{\text{在 } p \text{ 个处理器上 每单位时间完成的工作量}}{\text{在 } 1 \text{ 个处理器上 每单位时间完成的工作量}}$$

固定内存可扩展问题

- 大规模 N-Body 问题

- 获得 2012 年戈登-贝尔奖：
 - 模拟 1,073,741,824,000 粒子问题
 - 在日本 K 计算机上的计算

- 超大规模机器学习

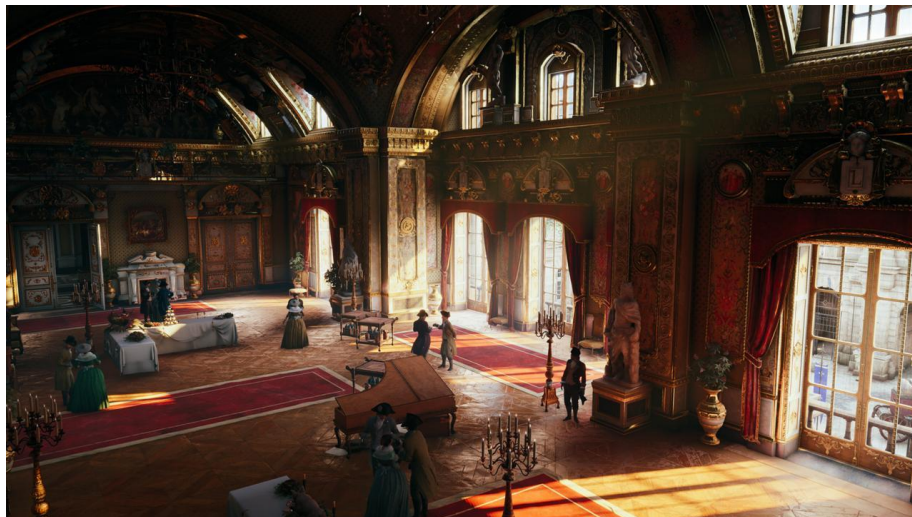
- 预训练语言模型（万亿或更大规模参数量）

- Memcached（在内存中缓存网页文件）

- 更多服务器 = 更多内存

皮克斯动画影业的可扩展性问题

- 渲染电影中的一个镜头（一系列帧）
 - 目标：最小完成时间（**固定问题规模**）
 - 将每帧分发到集群中的不同计算节点
- 艺术家对一个场景进行调光
 - 提供交互式的渲染（**固定执行时间**）
 - 更高的性能 = 更好的交互体验
- 物理模拟（比如液体模拟）
 - 并行的在多个计算节点上进行模拟，尽量占满所有内存（**固定内存可扩展问题**）
- 最终渲染一部电影
 - 场景复杂性受限于可用的内存容量

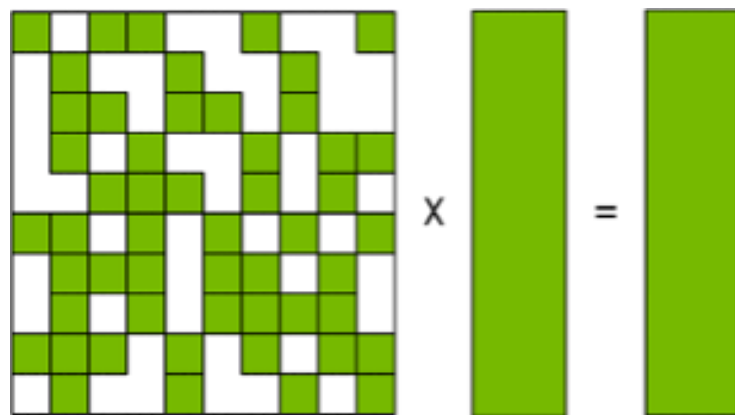


总结

- 任务分配
 - 静态、半静态、动态
 - 负载均衡: SpMV 优化
- 通信优化
 - 通信开销
 - 通信重叠
 - 通信拥塞
 - 算术密度
- 可扩展性
 - 加速比
 - 强扩展、弱扩展
 - 阿姆达尔定律

大作业三

- **稀疏矩阵矩阵乘法 (SpMM)**
 - 广泛应用于科学计算、机器学习应用
 - 需要稀疏结构和稠密结构的 Co-Design 与优化
- **稀疏计算**
 - 只计算非零元，节省计算量
 - 稀疏结构不规则，难以高效执行
 - 负载均衡，局部性，overhead
- **任务**
 - 使用 OpenMP 实现 CPU 多线程 SpMM
 - 使用 CUDA 实现 GPU SpMM
 - 在稀疏数据集测试性能并和baseline进行对比
- **评分标准**
 - 得分依据性能 (每个数据集单独评分) 和实验报告
 - **禁止抄袭**



SpMM 示意图